
Game-Theoretic Co-Evolution for LLM-Based Heuristic Discovery

Xinyi Ke^{1 2} Kai Li^{1 2} Junliang Xing³ Yifan Zhang^{1 2 4} Jian Cheng^{1 2 5}

Abstract

Large language models (LLMs) have enabled rapid progress in automatic heuristic discovery (AHD), yet most existing methods are predominantly limited by static evaluation against fixed instance distributions, leading to potential overfitting and poor generalization under distributional shifts. We propose Algorithm Space Response Oracles (ASRO), a game-theoretic framework that reframes heuristic discovery as a program level co-evolution between solver and instance generator. ASRO models their interaction as a two-player zero-sum game, maintains growing strategy pools on both sides, and iteratively expands them via LLM-based best-response oracles against mixed opponent meta-strategies, thereby replacing static evaluation with an adaptive, self-generated curriculum. Across multiple combinatorial optimization domains, ASRO consistently outperforms static-training AHD baselines built on the same program search mechanisms, achieving substantially improved generalization and robustness on diverse and out-of-distribution instances.

1. Introduction

Combinatorial Optimization (CO) underpins critical applications across engineering, autonomous systems, and large-scale resource management, yet its NP-hard nature makes exact algorithms infeasible at realistic scales (Papadimitriou & Steiglitz, 1998). Modern solvers therefore rely on hand-crafted heuristics and metaheuristics (Blum & Roli, 2003; Burke et al., 2009; 2010a; Drake et al., 2020). This manual design process is slow, brittle, and biased toward the structural assumptions of human experts, which often leads to severe performance degradation under distributional or

structural shift (Bengio et al., 2021; Manchanda et al., 2022).

The emergence of Large Language Models (LLMs) capable of synthesizing executable programs (Austin et al., 2021) has opened a promising direction for Automatic Heuristic Design (AHD). Recent work leverages this capability to automate the synthesis of executable heuristic solvers, enabling rapid construction of competitive solvers and partially automating the manual algorithm-design pipeline.

In practice, existing LLM-AHD pipelines adopt a closed-loop generate-evaluate-refine workflow: the LLM proposes heuristic programs, their execution on a fixed evaluator yields performance and behavioral feedback, and this feedback guides subsequent refinement. This iterative loop forms the backbone of current LLM-AHD methodologies (Romera-Paredes et al., 2024; Liu et al., 2024a; Ye et al., 2024; Novikov et al., 2025). Despite their empirical success, these frameworks are fundamentally tied to static evaluation (Liu et al., 2024a): heuristics are optimized against a fixed evaluation distribution, e.g., pre-designed datasets.

This static setting raises two core limitations. (i) It invites overfitting and fragility: discovered heuristics optimized on a fixed evaluator often generalize poorly under distributional shift (Sim et al., 2025). (ii) It induces a performance ceiling: once a heuristic is near-optimal, the evaluator fails to expose new weaknesses, hindering further improvement. Even when moving beyond static evaluation, existing approaches often rely on hand-crafted curricula or ad hoc difficulty schedules, leaving instance adaptation outside the core optimization objective and structurally retaining a single-agent training paradigm. Consequently, solver-instance interactions lack a principled, unified optimization framework, making stable and sustained co-adaptation difficult.

Motivated by this, we propose and formalize a game-theoretic, co-evolutionary formulation of LLM-based heuristic discovery, termed the Algorithm Space Response Oracle (ASRO). In ASRO, solvers and instance generators are modeled as two players in a zero-sum game, with strategies represented as executable programs¹. Solvers map instances to solutions, while generators induce instance distributions that

¹C²DL, Institute of Automation, Chinese Academy of Sciences ²School of Artificial Intelligence, University of Chinese Academy of Sciences ³Tsinghua University ⁴University of Chinese Academy of Sciences, Nanjing ⁵AiRiA. Correspondence to: Kai Li <kai.li@ia.ac.cn>.

¹Throughout the paper, we study algorithms through their executable program realizations. For notational simplicity, we refer to these realizations as *programs*, which serve as concrete, executable solvers or generators synthesized by the LLM.

challenge solver performance. ASRO maintains growing pools of strategies on both sides, and iteratively (i) evaluates all solver-generator pairs to construct a payoff matrix, (ii) computes mixed meta-strategies (i.e., probability distributions over the current strategy pools), and (iii) invokes LLM-based best-response oracles to synthesize new solver or generator programs against the opponent’s meta-strategy.

Conceptually, ASRO draws inspiration from Policy Space Response Oracles (PSRO) (Lanctot et al., 2017), sharing its population-based, best-response-driven optimization paradigm. Unlike PSRO, which operates in parametric policy spaces and approximates best responses via reinforcement learning, ASRO works directly in discrete program space, where strategies are executable solver and instance generator programs synthesized by LLMs. A key advantage of this formulation is that generator programs induce structured stochastic families of instance distributions. By incorporating generators into the meta-game, ASRO optimizes solver populations against evolving instance distributions rather than fixed datasets. ASRO induces a self-generated curriculum: generators adaptively expose weaknesses of the current solver mixture, and solvers in turn adapt to increasingly hard and diverse instance distributions.

Our main contributions are as follows:

- **Game-theoretic framework for LLM-based AHD.** We formulate LLM-driven heuristic discovery as a two-player zero-sum game over executable programs, replacing static evaluation with a PSRO-style “solve-and-expand” loop in discrete program space.
- **Meta-game driven co-evolution.** Persistent strategy pools and equilibrium-based meta-strategies induce sustained solver-generator co-evolution, avoiding short-horizon adversarial dynamics.
- **An oracle-agnostic, program-space framework.** ASRO specifies a game-theoretic optimization structure in program space while leaving the choice of program search mechanism open.
- **Empirical validation across representative CO domains.** ASRO consistently outperforms static-training counterparts built on identical program search mechanisms, showing improved robustness and generalization under substantial scale and structural variation.

2. Related Works

2.1. Automated Heuristic Design

Early AHD relied on hyper-heuristics and genetic programming, evolving heuristics expressed in manually designed domain-specific languages (DSLs) (Burke et al., 2003; 2013;

2018; Pillay & Qu, 2018). A major line of work develops heuristic expressions via Genetic Programming (GP), spanning early program-evolution frameworks (Koza, 1994; Burke et al., 2009; 2010b) and more recent applications to routing and packing problems (Hildebrandt et al., 2010; Duflo et al., 2019). These methods are often paired with automatic configuration frameworks such as SMAC and irace (Hutter et al., 2011; López-Ibáñez et al., 2016), but their low-level search spaces limit scalability and prevent the exploitation of high-level algorithmic structures.

The integration of LLMs into AHD has transformed the field, enabling the synthesis of executable heuristics through semantic reasoning. A prominent paradigm is evaluation-guided, iterative program evolution, exemplified by a range of representative works, including early frameworks such as AEL (Liu et al., 2023b) and FunSearch (Romera-Paredes et al., 2024), as well as EoH (Liu et al., 2024a) and more recent systems like AlphaEvolve (Novikov et al., 2025). Meanwhile, works like (Ye et al., 2024; Liu et al., 2025b) further enrich this process by incorporating verbal reflection and historical experience to guide improvement, reflecting a broader trend toward iterative self-refinement (Madaan et al., 2023; Shinn et al., 2023). Expanding the scope further, research has investigated the automatic generation and evolution of metaheuristic algorithms and operators using LLMs across various optimization paradigms (Pluhacek et al., 2023; van Stein & Bäck, 2024; Hemberg et al., 2024; Dat et al., 2025; Yao et al., 2025; Surina et al., 2025). However, these approaches remain predominantly evaluator-static, optimizing against predefined task distributions and overlooking the potential of co-evolutionary adaptation.

2.2. Generalization and Robustness in Combinatorial Optimization

Generalization has long been a central focus in Neural Combinatorial Optimization (NCO). Prior work explores curriculum learning (Lisicki et al., 2020; Zhang et al., 2022; Iklakov et al., 2023; Liu et al., 2024b), distributional diversification (Jiang et al., 2022; Zhou et al., 2023; Luo et al., 2023), and hardness-aware or adversarial sampling (Zhang et al., 2022), including game-theoretic adversarial variants (Wang et al., 2022; 2024). Furthermore, meta-learning (Manchanda et al., 2022; Qiu et al., 2022; Wang & Li, 2023; Son et al., 2023; Chen et al., 2023) and knowledge distillation (Bi et al., 2022; Zhang et al., 2023; Zheng et al., 2025) have been widely adopted to enhance robustness. Notably, these lines of work primarily operate by optimizing continuous parameters within neural architectures, rather than evolving symbolic, executable programs.

Recent LLM-AHD studies examine generalization via heuristic set diversification, cross task transfer, and problem agnostic modeling (Jiang et al., 2025; Chen et al., 2025; Shi

et al., 2026; Liu et al., 2026). Most, however, rely on static distributions. While EALG introduces an initial GAN-style solver–generator loop (Duan et al., 2025), its self-play formulation does not explicitly maintain a persistent strategy set on either side. This makes the dynamics more prone to cycling (Czarnecki et al., 2020) and limits the ability to form a stable, equilibrium-driven curriculum across diverse instance families, which is precisely the gap that our ASRO framework aims to address.

2.3. Game-Theoretic Response Frameworks

Game-theoretic response frameworks comprise a family of methods that iteratively expand a game’s strategy sets by solving restricted games and introducing best responses. Classical approaches like Fictitious Play, Double Oracle, and empirical game-theoretic analysis established this foundation (Brown, 1951; McMahan et al., 2003; Wellman, 2006; Shoham & Leyton-Brown, 2008), while PSRO later instantiated these principles in deep multi-agent reinforcement learning by maintaining and updating sets of policies via iterative best-response training (Lanctot et al., 2017). This line of work was subsequently refined by improved meta-solvers and ranking mechanisms—exemplified by Alpha-PSRO (Muller et al., 2020) and Nash averaging (Balduzzi et al., 2018; 2019)—and scaled to complex environments via AlphaStar’s league-based training (Jaderberg et al., 2017; Vinyals et al., 2019). In contrast to these approaches, which operate on parametric neural policies, our work adapts the response-oracle paradigm to the domain of program synthesis, where solver logic and instance generators co-evolve as executable programs.

3. ASRO: A Program Space Co-Evolution Framework for AHD

3.1. Preliminaries: Policy-Space Response Oracles

PSRO (Lanctot et al., 2017) is a response-based framework for N -player games that operates by iteratively expanding a restricted set of strategies $\Pi_i^{(t)} \subset \Pi_i$ for each player $i = 1, \dots, N$. The procedure follows an iterative “solve-and-expand” workflow. At iteration t , a **meta-strategy** $\sigma_i^{(t)}$ is computed by solving the game restricted to the current strategy sets, where $\sigma_i^{(t)}$ is a probability distribution over $\Pi_i^{(t)}$ and $\sigma_{-i}^{(t)}$ denotes the joint meta-strategy of all players except player i . Subsequently, a Best-Response (BR) oracle expands the strategy set by finding

$$\pi_{i,\text{new}} = \arg \max_{\pi_i \in \Pi_i} \mathbb{E}_{\pi_{-i} \sim \sigma_{-i}^{(t)}} [u_i(\pi_i, \pi_{-i})], \quad (1)$$

where $u_i(\pi_i, \pi_{-i})$ is the expected utility of player i . In PSRO, strategies are typically parameterized by neural policies, with BRs approximated via reinforcement learning. Details are provided in Appendix A.

3.2. A Program-Space Game Formulation of AHD

Our key observation is that AHD implicitly defines an interaction between two programmatic choices: a solver program that determines how solutions are constructed for a CO instance, and an instance-generation program that determines which instances are used for evaluation. Based on this observation, we introduce the ASRO framework, which extends PSRO from policy spaces to the space of symbolic, executable programs.

Strategy Spaces. The **Solver Space** $\mathcal{S}$ contains solver programs s , each of which specifies heuristic decision logic that is executed within a fixed, domain-specific solver procedure to construct a feasible solution for a given instance x . The **Generator Space** $\mathcal{G}$ contains generator programs g that specify stochastic instance distributions; sampling $x \sim g(\cdot)$ yields structurally valid problem instances generated from the program’s specification.

Payoff Function. We model the interaction between solver and generator programs as a two-player zero-sum game defined over a task-specific *normalized reference gap*, a standard performance measure in CO (Beasley, 1990). For a solver program s and an instance x , let

$$\text{gap}(s, x) = \frac{V(s, x) - v^*(x)}{v^*(x)} \quad (2)$$

denote the normalized gap with respect to a reference value $v^*(x)$, where $V(s, x)$ is the objective value produced by solver s on instance x , and $v^*(x)$ denotes a task-dependent reference value. Smaller (and potentially negative) gaps indicate better solver performance. The payoff of a solver–generator pair (s, g) is defined as the expected gap under the generator-induced instance distribution:

$$U(s, g) = \mathbb{E}_{x \sim g} [\text{gap}(s, x)]. \quad (3)$$

The solver seeks to minimize $U(s, g)$, while the generator seeks to maximize it.

3.3. Meta-Strategy Computation

Given the current solver and generator strategy pools $\mathcal{S}^{(t)}$ and $\mathcal{G}^{(t)}$, ASRO constructs a restricted payoff matrix $M^{(t)}$, where each entry is

$$M_{ij}^{(t)} = U(s_i, g_j). \quad (4)$$

Since solvers minimize gap while generators maximize it, the interaction forms a finite zero-sum matrix game. At iteration t , ASRO computes mixed meta-strategies $(\sigma_s^{(t)}, \sigma_g^{(t)})$ by approximately solving the minimax equilibrium

$$\min_{\sigma_s} \max_{\sigma_g} \mathbb{E}_{s \sim \sigma_s, g \sim \sigma_g} [M_{sg}^{(t)}]. \quad (5)$$

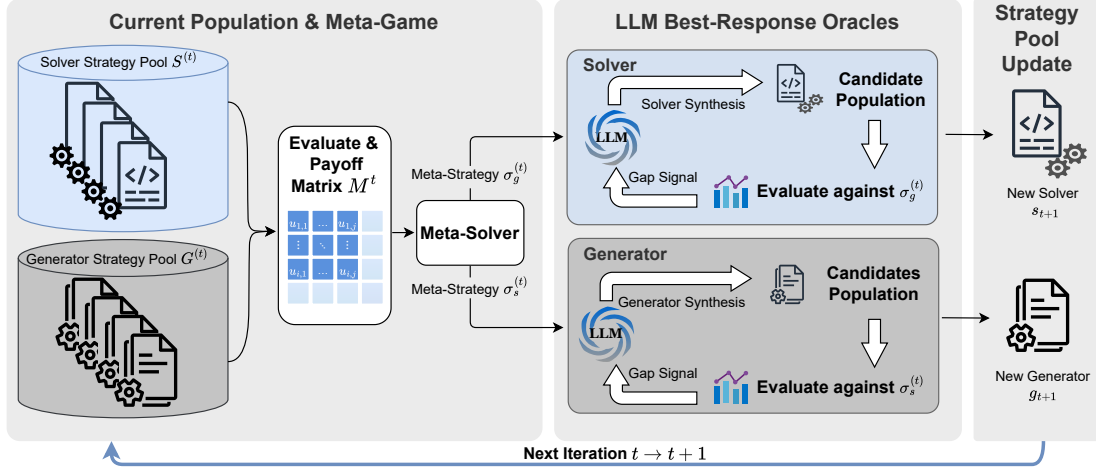


Figure 1. Overview of ASRO. The iterative process is structured into three phases. (Left) The current solver strategy pool $S^{(t)}$ and generator strategy pool $G^{(t)}$ are evaluated pairwise to compute a payoff matrix $M^{(t)}$. A meta-solver then computes mixed meta-strategies $(\sigma_s^{(t)}, \sigma_g^{(t)})$ over the existing strategy pools. (Middle) Program-space best-response oracles synthesize new solver and generator programs conditioned on the opponent’s current meta-strategy, instantiated by an underlying program search mechanism (see Figure 2). (Right) The newly discovered solver s_{t+1} and generator g_{t+1} are added to their respective strategy pools for the next iteration ($t \rightarrow t + 1$), enabling systematic exploration of the program space.

For finite zero-sum games, this equilibrium can be obtained via the standard linear-program formulation of the minimax problem or, equivalently, via no-regret dynamics such as multiplicative weights (Freund & Schapire, 1997). The resulting meta-strategies provide opponent distributions for the subsequent best-response synthesis.

3.4. Program-Space Best-Response Oracle

To optimize the best-response objective in Eq. (1) over executable programs, ASRO assumes access to a *program-space best-response oracle*, implemented by a *program search mechanism*. Given the opponent’s meta-strategy, this oracle approximately optimizes the induced best-response objective and returns a corresponding program, referred to as an *approximate best response* (ABR). When the responding player is a solver or a generator, we write the returned programs as $ABR_s(\sigma_g)$ and $ABR_g(\sigma_s)$, respectively. ASRO is agnostic to how this search is performed, enabling modular instantiations. We denote an instantiation using mechanism X as *ASRO-X*. X may correspond to a mechanism derived from or inspired by existing LLM-based algorithm discovery methods, without inheriting their original optimization objectives or training pipelines.

Instantiation with Evolutionary Search (ASRO-EoH). In this work, ASRO instantiates the best-response oracle via a bounded evolutionary program search derived from Evolution of Heuristics (EoH) (Liu et al., 2024a), referred to as *ASRO-EoH*; an overview is shown in Figure 2. The oracle maintains a population of K heuristic programs, which is initialized at the first ASRO iteration and warm-started from

previously discovered programs thereafter. Within a single oracle call, the population evolves for a fixed number of R rounds through iterative proposal, evaluation, and selection over candidate programs. New candidates are synthesized via structured LLM-based exploration and mutation operators, followed by a selection step that balances performance and population diversity. After the search terminates, the highest-performing program is returned as an approximate best response and used to expand the ASRO strategy pool. Implementation details are provided in Appendix B.

3.5. Training Procedure and Analysis

ASRO (Figure 1) is an iterative framework that alternates between solving a restricted program-level meta-game and expanding the strategy pools via LLM-based approximate best responses. At each iteration, ASRO updates the payoff matrix over the current solver and generator strategy pools, computes equilibrium meta-strategies, and invokes best-response oracles to synthesize new solver and generator programs. The full procedure is summarized in Algorithm 1.

4. Experiments

4.1. Experimental Setup

Common Settings. We use **DeepSeek-V3.2** (Liu et al., 2025a) as the backbone LLM for both solver and generator synthesis, with a temperature of 1, promoting exploration in program space. The co-evolutionary process runs for $T = 8$ iterations. During training, a small portion of instances is always drawn from a fixed, domain-specific *base generator*

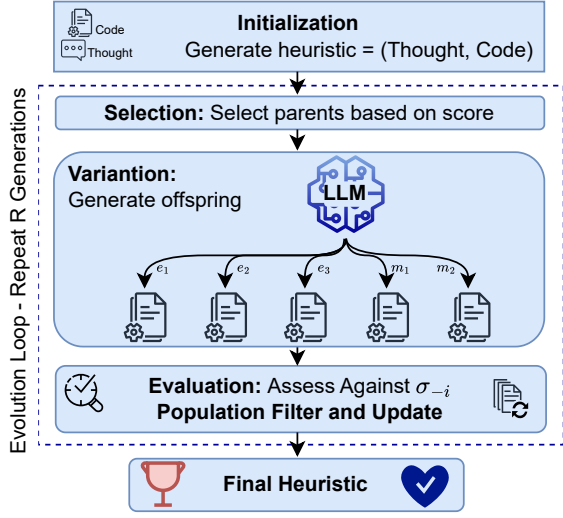


Figure 2. Program-space best-response oracle in ASRO, instantiated via an EoH-style evolutionary program search. Under a fixed opponent meta-strategy σ_{-i} provided by ASRO, the oracle performs a bounded evolutionary search over programs in the corresponding strategy space. Candidate programs are initialized, explored, and mutated using structured LLM-based operators, and evaluated according to the induced best-response objective.

Algorithm 1 ASRO: Program-Level Co-Evolution for AHD

- 1: **Input:** Initial pools $\mathcal{S}^{(0)}, \mathcal{G}^{(0)}$; iteration horizon T
- 2: **for** $t = 0$ to $T - 1$ **do**
- 3: (1) Meta-Game Evaluation
- 4: Update payoff matrix $M^{(t)}$ over the strategy pools
- 5: (2) Meta-Strategy Computation
- 6: $(\sigma_s^{(t)}, \sigma_g^{(t)}) \leftarrow \text{METASOLVER}(M^{(t)})$
- 7: (3) LLM-Based Approximate Best Responses
- 8: $s_{\text{new}} \leftarrow \text{ABR}_s(\sigma_g^{(t)})$ (solver BR)
- 9: $g_{\text{new}} \leftarrow \text{ABR}_g(\sigma_s^{(t)})$ (generator BR)
- 10: (4) Strategy Pool Expansion
- 11: $\mathcal{S}^{(t+1)} \leftarrow \mathcal{S}^{(t)} \cup \{s_{\text{new}}\}$
- 12: $\mathcal{G}^{(t+1)} \leftarrow \mathcal{G}^{(t)} \cup \{g_{\text{new}}\}$
- 13: **end for**
- 14: **Output:** Final solver and generator strategy pools

to stabilize early-stage co-evolution and avoid premature over-specialization

Framework-level comparison with EoH. ASRO-EoH and EoH use the same evolutionary program search mechanism; the difference is that EoH is trained on a fixed distribution induced by the base generator, whereas ASRO-EoH operates under a game-based framework with an evolving instance distribution induced by the generator strategy pool.

Evaluation Metric. Across all tasks, we evaluate performance using the reference gap defined in Eq. (2).

Convergence Metrics. We assess convergence using exploitability-based metrics estimated using approximate

best responses, which quantify the gain from a unilateral deviation. Given mixed strategies $(\sigma_s^{(t)}, \sigma_g^{(t)})$ at iteration t , the solver and generator exploitabilities are

$$\begin{aligned} \text{Exp}_s(t) &= u(\sigma_s^{(t)}, \sigma_g^{(t)}) - u(\text{ABR}_s(\sigma_g^{(t)}), \sigma_g^{(t)}), \\ \text{Exp}_g(t) &= u(\sigma_s^{(t)}, \text{ABR}_g(\sigma_s^{(t)})) - u(\sigma_s^{(t)}, \sigma_g^{(t)}), \end{aligned} \quad (6)$$

We report the Approximate NashConv (ANC), $\text{ANC}(t) = \text{Exp}_s(t) + \text{Exp}_g(t)$, which summarizes the exploitability of the current solver-generator mixture (Timbers et al., 2022).

4.2. Domains and Implementation Details

We evaluate ASRO on three well-studied combinatorial optimization problems: online bin packing (OBP), the traveling salesman problem (TSP), and the capacitated vehicle routing problem (CVRP), spanning online and offline optimization as well as diverse decision structures. Hyperparameter settings and LLM usage are summarized in Appendix D, while task-specific formulations, prompt templates, and domain details are provided in Appendix E, F, and G. Code for reproducing our results is available at <https://github.com/Kexy24/ASRO>.

4.2.1. ONLINE BIN PACKING (OBP)

Problem Setting. We study online bin packing problem (Johnson et al., 1974; Garey & Johnson, 1981), where items arrive sequentially and must be assigned irrevocably to bins of capacity C , with the objective of minimizing the total number of bins used (Seiden, 2002; Balogh et al., 2014).

Solver Representation. Solver programs are executed within a fixed online packing framework. Each solver program specifies an online greedy decision rule in the form of a numerical priority function that assigns a score to each feasible bin upon item arrival (Ramanan et al., 1989). The packing process sequentially handles arriving items and enforces feasibility; all other components of the procedure are kept fixed, with items placed into the feasible bin with the highest score, without lookahead or post-processing.

Reference Values. $v^*(x)$ are obtained using OR-Tools CP-SAT when optimality can be certified (Google OR-Tools Team, 2024); otherwise, we use the Martello–Toth lower bound (Martello & Toth, 1990).

Benchmarks and Arrival Orders. We evaluate on widely used public bin packing benchmarks (Delorme et al., 2018): Falkenauer T and U families (Falkenauer, 1996), the Hard28 benchmark (Delorme et al., 2018), and additional instances drawn from a range of Weibull distributions (Castiñeiras et al., 2012). These benchmarks cover both classical test cases and structurally challenging settings. The base generator used during training follows a Weibull distribution. We evaluate under multiple standard arrival orders (random, ascending), fixed independently of the algorithms.

Baselines. We compare ASRO against canonical online heuristics (First Fit (Ullman, 1971; Dósa & Sgall, 2013) and Best Fit (Johnson et al., 1974; Albers et al., 2021)) and EoH (Liu et al., 2024a), an LLM-based solver-only baseline.

4.2.2. TRAVELING SALESMAN PROBLEM (TSP)

Problem Setting. We consider the Euclidean Traveling Salesman Problem with N cities, where the objective is to find a Hamiltonian tour of minimum total length.

Solver Representation. Solver programs operate within a fixed Guided Local Search (GLS) framework (Voudouris & Tsang, 1999; Liu et al., 2025b). Each solver specifies an edge-distance update rule, producing a modified distance matrix for evaluating candidate moves. All other components of GLS, including the search loop and standard local search operators (e.g., 2-opt, relocate), are kept fixed, while the learned update reshapes the search landscape.

Reference Values. $v^*(x)$ are obtained using the Concorde solver for all instances considered (Applegate et al., 2011).

Benchmarks. Evaluation is conducted on held-out uniformly sampled 100-node instances and TSPLIB (Reinelt, 1991), covering homogeneous Euclidean cases and structured instances with diverse geometric properties. The base generator samples instances uniformly.

Baselines. We compare ASRO against classical constructive heuristics (Nearest and Farthest Insertion; NI, FI) (Rosenkrantz et al., 1977) and EoH (Liu et al., 2024a).

4.2.3. CAPACITATED VEHICLE ROUTING PROBLEM

Problem Setting. We study the CVRP, where a fleet of identical vehicles departs from and returns to a single depot to serve customers with known demands. Each customer must be visited exactly once, and the total demand served by any vehicle cannot exceed the vehicle capacity Q . The objective is to minimize the total travel cost over all routes.

Solver Representation. Solver programs are executed within a fixed greedy route-construction procedure. Each solver program specifies a numerical priority function that assigns a score to each currently feasible customer at a routing step. The customer with the highest score is selected and appended to the current route.

Reference Values. Reference values $v^*(x)$ are obtained using PyVRP (Wouda et al., 2024) under a fixed time budget, serving as high-quality heuristic reference solutions and not necessarily global optima.

Benchmarks. We evaluate on CVRP benchmark instances from CVRPLIB (Augerat et al., 1995; Uchoa et al., 2017), including the A, B, E, F, M, P, and X families. These instance families span a broad range of problem sizes and

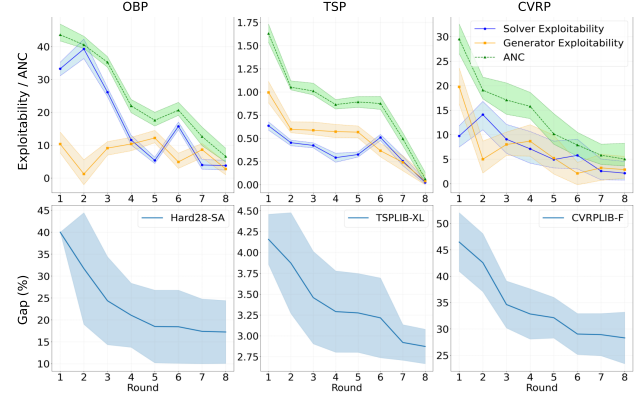


Figure 3. ASRO training dynamics across combinatorial optimization tasks. (Top) Convergence behavior of ASRO on OBP, TSP, and CVRP, measured by solver exploitability, generator exploitability, and ANC over iterations, shown for a representative ASRO run; shaded regions denote confidence intervals via bootstrap resampling over instances (20 per generator). (Bottom) Stability across random runs: test gap (%) (mean $\pm$ std) over five independent runs on the hardest benchmark for each task, where benchmark difficulty is determined by the EoH baseline ranking.

structural characteristics, covering both relatively simple and highly challenging routing scenarios. The base generator samples customer coordinates uniformly from $[0, 100]^2$ and customer demands uniformly from $[1, Q/3]$.

Baselines. We compare ASRO with several classical CVRP heuristics, including nearest-neighbor construction with local improvement (NN+2-opt) and parallel insertion heuristics (PI) (Liu et al., 2023a), as well as EoH (Liu et al., 2024a).

4.3. Results

Figure 3 summarizes the empirical training dynamics and stability of ASRO across different combinatorial optimization tasks. Across domains, ANC exhibits a general downward trend over iterations, while solver and generator exploitability remain bounded. Meanwhile, performance, measured by the optimality gap on test benchmarks, improves consistently across random runs despite stochastic LLM synthesis and approximate best-response updates. Task-specific differences are evident: OBP exhibits stronger local fluctuations due to its discrete and non-smooth payoff structure; TSP shows comparatively smoother dynamics consistent with its geometric formulation and local search procedures; and CVRP admits greater room for solver-generator adaptation, reflecting its intrinsic structure induces a non-trivial trade-off between minimizing route cost and maintaining capacity feasibility, which gives rise to more persistent adaptive interactions without destabilizing overall convergence.

OBP Table 1 summarizes the performance on standard online bin packing benchmarks. Across all settings, ASRO consistently outperforms the EoH baseline. As the instance

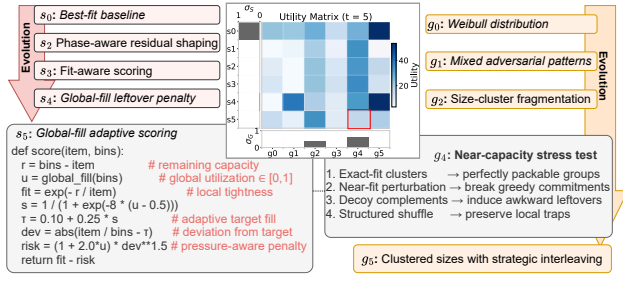


Figure 4. A snapshot of solver-generator co-evolution in online bin packing. The figure shows a utility matrix at Round 5, with meta-strategies computed from Round 4. Side panels summarize the solver and generator strategies present in the pool at Round 4. In this snapshot, the generator-side meta-strategy assigns dominant weight to g_4 , and the solver s_5 is generated as a best response under this adversarial pressure. The dashed connection indicates that s_5 is generated as a best response to g_4 . The highlighted entry (red box) shows the utility of s_5 evaluated against g_4 .

Table 1. Main OBP results (gap % $\downarrow$). Gap to the lower bound on standard OBP benchmarks. F-T and F-U denote the Falkenauer random-arrival sets. Hard28-R and Hard28-SA correspond to random and size-ascending arrivals on the Hard28 benchmark.

Method	F-U	F-T	Hard28-R	Hard28-SA	Weibull
Best Fit	5.39	12.66	8.45	40.02	1.75
First Fit	6.21	12.63	9.88	40.02	2.02
EoH	5.00	12.11	8.40	40.02	1.56
ASRO-EoH	4.53	7.11	8.06	12.45	1.20

structure becomes more challenging—from the simpler Falkenauer U set, to the more structured Falkenauer T set, and further to the Hard28 benchmark—the performance gap between ASRO and EoH tends to increase. This trend suggests that adversarial co-evolution yields greater benefits on harder and more structured instances. Importantly, even on the Weibull distribution used to train EoH, ASRO does not exhibit performance degradation, suggesting improved robustness without sacrificing in-distribution performance.

Figure 4 provides a view of how ASRO induces solver adaptation through generator pressure. The generator meta-strategy concentrates on g_4 , which constructs item sequences that appear easy to pack early on but systematically result in fragmented bin capacity at later stages. In this setting, such residual-trap structures are particularly harmful to locally greedy solvers, whose irrevocable early decisions leave little flexibility for future items. Facing this concentrated stress pattern, the solver-side oracle synthesizes s_5 as an approximate best response. Compared to earlier heuristics, s_5 appears to trade off immediate fit against future leftover risk, favoring packing decisions that preserve global flexibility under near-capacity conditions. This interaction provides an illustrative example of the core mechanism of ASRO: generators expose concrete failure

Table 2. Main TSP results (gap % $\downarrow$). Average optimality gap on a synthetic uniform distribution and on TSPLIB instances grouped by size: LIB-S ($n \leq 200$), LIB-M ($201 \leq n \leq 500$), LIB-L ($501 \leq n \leq 1000$), and LIB-XL ($n > 1000$).

Method	LIB-S	LIB-M	LIB-L	LIB-XL	Uniform
NI	19.84	23.51	23.03	21.22	20.88
FI	7.30	10.49	13.30	15.19	8.54
EoH	0.77	1.99	3.40	4.20	0.27
ASRO-EoH	0.21	0.49	1.43	3.00	0.05

Table 3. Main CVRP results (gap % $\downarrow$). Average optimality gap across CVRPLib instance families (A, B, E, F, M, P, and X).

Method	A	B	E	F	M	P	X
NN + 2-opt	35.58	41.30	34.81	46.53	47.14	28.24	22.71
PI	41.82	39.39	63.07	43.73	95.15	51.48	58.34
EoH	20.19	12.22	41.83	59.78	54.22	23.68	25.71
ASRO-EoH	16.85	10.49	20.21	28.71	22.45	19.45	13.65

modes of the solver mixture, and solvers adapt by internalizing defenses against these modes in program space.

TSP Table 2 shows that ASRO consistently outperforms the EoH baseline on both simple uniform instances and more complex TSPLIB benchmarks. Moreover, as problem size increases, the performance gap between ASRO and EoH widens, indicating stronger scalability and improved generalization to larger instances, even though both methods are trained with generators limited to the same problem size. On several instances, including PR107, PR136, PR144, TSP225, U159, and KROB100, ASRO attains small negative optimality gaps relative to reported best-known solution (BKS) values for TSPLIB instances (Reinelt, 1991), suggesting highly competitive solution quality under the same evaluation protocol. Overall, these results demonstrate that ASRO achieves robust and scalable performance across TSP instances of varying structural complexity and problem size.

CVRP Table 3 reports average optimality gaps on CVRPLib instance families. While A and B consist of relatively regular and well-structured instances, E, F, M, P, and X exhibit substantially higher heterogeneity in customer distributions, route lengths, and capacity profiles. ASRO achieves the lowest gap across all families, performing strongly on both simpler and more complex instances, indicating robust performance across diverse CVRP instance structures.

4.4. Ablation Study

Data Augmentation and Self-play. To examine whether the gains of ASRO can be attributed to stronger training data or to adversarial interaction alone, we compare ASRO

Table 4. **Ablation on training strategies for TSP (gap % ↓).** Comparison between EoH with data augmentation (DA) or self-play (SP) and ASRO-EoH.

Method	LIB-S	LIB-M	LIB-L	LIB-XL	Uniform
EoH	0.77	1.99	3.40	4.20	0.27
EoH + SP	0.30	1.06	2.22	3.50	0.22
EoH + DA	0.44	1.17	2.03	3.66	0.28
ASRO-EoH	0.23	0.67	1.73	2.82	0.06

against two EoH-based baselines: (i) EoH with data augmentation (DA) and (ii) an EoH self-play variant. In both cases, the program search mechanism is identical to ASRO’s, ensuring that differences arise from instance distributions or interaction dynamics rather than from search.

For DA, the solver is trained on a fixed mixture of geometric instance families, including uniform, clustered, grid-like, ring-shaped, and multi-scale distributions, providing substantially richer distributional diversity than single-distribution training. Full sampling details are provided in Appendix H.1. The self-play baseline removes the game-theoretic structure of ASRO while retaining solver-generator optimization. At each iteration, the solver and generator are optimized against each other, but no historical strategies are retained. In ASRO terminology, this corresponds to discarding the strategy pool and restricting both players to their most recent strategies, resulting in a greedy, memoryless adversarial loop.

As shown in Table 4, both data augmentation and self-play yield clear improvements over standard EoH, indicating that increased distributional diversity and direct adversarial interaction are beneficial for heuristic learning. However, both variants remain inferior to ASRO, especially on larger and more structured TSPLIB instances. This suggests that while these factors contribute to stronger solvers, they are insufficient on their own: the persistent strategy pool and meta-game dynamics in ASRO play a key role in maintaining sustained pressure and robust co-evolution.

Effect of LLM Backbones. Table 5 reports ASRO-EoH with different LLM backbones for program search. While stronger backbones achieve modestly lower gaps on TSPLIB instances, smaller models remain effective. Notably, ASRO-EoH with LLaMA-3-8B (Grattafiori et al., 2024) outperforms the EoH baseline trained with a stronger backbone (DeepSeek) on most evaluation settings, indicating that ASRO’s gains primarily stem from solver-generator co-evolution rather than backbone scale.

Oracle-agnosticity of ASRO. All main experiments instantiate the program-space best-response oracle using an EoH-style program search mechanism; however, ASRO does not depend on any particular choice of search procedure. Replacing the EoH-style oracle with a ReEvo-style search

Table 5. **Effect of different LLM backbones in ASRO (TSP) (gap % ↓).** All variants use the same ASRO-EoH framework and differ only in the LLM used for program-space search; the **EoH** baseline (DeepSeek) is included for reference.

LLM	LIB-S	LIB-M	LIB-L	LIB-XL	Uniform
DeepSeek	0.23	0.67	1.73	2.82	0.06
Gemini	0.23	0.45	0.96	2.45	0.06
GPT-5 nano	0.26	0.69	2.38	3.68	0.10
GPT-5 mini	0.21	0.47	1.39	2.53	0.10
LLaMA-3-8B	0.31	1.01	3.00	4.33	0.20
EoH (DeepSeek)	0.77	1.99	3.40	4.20	0.27

Table 6. Comparison between vanilla ReEvo and ASRO with the same ReEvo-style search mechanism (gap % ↓).

Method	LIB-S	LIB-M	LIB-L	LIB-XL	Uniform
ReEvo	0.38	1.35	2.25	3.65	0.38
ASRO-ReEvo	0.30	1.19	2.06	3.53	0.07

mechanism (Ye et al., 2024) consistently improves over vanilla ReEvo on TSPLIB (Table 6), indicating that ASRO’s gains primarily arise from its response-based evaluation structure rather than from any particular search mechanism.

Computation Cost Analysis. ASRO introduces additional computational overhead due to best-response searches on both sides of the game and repeated solver-generator interaction evaluations. However, these evaluations are highly parallelizable in practice and are executed in a batched manner, substantially reducing the effective wall-clock overhead (Appendix C). To better characterize this, we report a comparison between ASRO and EoH under the same wall-clock budget in Appendix Table 10. Under this constraint, ASRO consistently achieves lower optimality gaps across all benchmark regimes. One key factor is that EoH converges quickly under a fixed instance distribution, resulting in diminishing returns, whereas ASRO maintains additional improvement capacity through alternating solver and generator updates. We additionally observe that the extra overhead primarily arises from program evaluations rather than LLM inference, while ASRO also uses fewer LLM API calls than EoH under the same wall-clock budget (Appendix Table 11).

Transfer Across Solver Procedures. In ASRO, solver strategies are represented as executable heuristic programs embedded within fixed solver procedures. Consequently, performance depends not only on the discovered heuristics themselves, but also on the underlying solver procedures in which they are instantiated. To evaluate whether ASRO generalizes across different solver procedures, we instantiate ASRO within the PyVRP (Wouda et al., 2024) framework by discovering an edge-weight transformation heuristic for PyVRP. Unlike the relatively simple greedy route-construction procedure used in the main CVRP experiments, PyVRP incorporates substantially more advanced components including population search, local improvement,

Table 7. **Transfer across solver procedures on CVRP (gap % ↓).** Average optimality gap across CVRPLib instance families (A, B, E, F, M, P, and X) under matched test-time budgets.

Method	A	B	E	F	M	P	X
KGLS	2.41	2.87	2.78	2.48	4.26	2.10	3.72
PyVRP	0.97	1.15	1.91	2.56	4.46	1.55	4.25
PyVRP (ASRO)	0.50	0.63	1.33	1.18	3.41	0.83	1.91

and ruin-and-repair mechanisms. We additionally include KGLS (Arnold & Sörensen, 2019), a strong knowledge-guided local-search heuristic, as a reference baseline. Table 7 shows that ASRO consistently improves PyVRP across all CVRPLib instance groups. These results suggest that the heuristics discovered by ASRO provide gains complementary to stronger solver procedures.

5. Discussion

Summary. This paper proposed ASRO, a game-theoretic framework for automatic heuristic discovery that replaces static evaluation with adaptive, interaction-driven assessment by modeling solver-generator interaction as a two-player zero-sum game with persistent strategy pools. We evaluated ASRO on three representative combinatorial optimization problems, including online bin packing, the traveling salesman problem, and the capacitated vehicle routing problem. Experimental results show that ASRO consistently discovers heuristics that outperform the EoH baseline, which is trained on fixed instance distributions using the same LLM-based search procedure, and demonstrates strong robustness across diverse and structured benchmark instances. In practice, ASRO trades additional computation for improved robustness, while its evaluation and search procedures remain naturally amenable to large-scale parallelization. Overall, ASRO establishes a principled game-theoretic framework that formulates heuristic discovery as an interactive process over executable programs, enabling robustness and generalization beyond fixed evaluation settings.

Limitations and future work. ASRO currently relies on reliable performance signals to compare generator strategies, yet for many combinatorial optimization instances exact optima are unavailable; in such cases, optimality gaps must be computed against strong but incomplete or time-limited oracles (e.g., CP-SAT bounds (Google OR-Tools Team, 2024) or feasible solutions from PyVRP (Wouda et al., 2024)). Such approximations introduce estimation noise into the meta-game and may obscure fine-grained distinctions among generators. Looking forward, ASRO can be extended beyond a strictly zero-sum formulation to multi-objective or regularized meta-games that explicitly trade off hardness, diversity, and realism for more controlled distribution shaping; it can also be extended to support teacher-student interactions, where generators serve as

adaptive curriculum designers rather than purely adversarial opponents; and more broadly, the program-space game perspective underlying ASRO may generalize beyond combinatorial optimization to settings such as algorithmic reasoning, planning, and symbolic decision-making.

Acknowledgements

This work is supported in part by the Strategic Priority Research Program of the Chinese Academy of Sciences (Grant No. XDA0480201), the Natural Science Foundation of China (Grant Nos. 62222606, 61902402 and 62273347), the Key Research and Development Program of Jiangsu Province (Grant No. BE2023016), and the CCF-NetEase ThunderFire Innovation Research Funding (Grant No. 202605).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Albers, S., Khan, A., and Ladewig, L. Best fit bin packing with random order revisited. *Algorithmica*, 83(9):2833–2858, 2021.
- Alsheddy, A., Voudouris, C., Tsang, E. P. K., and Alhindi, A. Guided local search. In *Handbook of Heuristics*, pp. 1–37. Springer International Publishing, Cham, 2016.
- Applegate, D. L., Bixby, R. E., Chvátal, V., and Cook, W. J. The traveling salesman problem: a computational study. In *The Traveling Salesman Problem*. Princeton University Press, 2011.
- Arnold, F. and Sörensen, K. Knowledge-guided local search for the vehicle routing problem. *Computers & Operations Research*, 105:32–46, 2019.
- Augerat, P., Naddef, D., Belenguer, J. M., Benavent, E., Corberan, A., and Rinaldi, G. Computational results with a branch and cut code for the capacitated vehicle routing problem. Technical report, 1995.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Balduzzi, D., Racaniere, S., Martens, J., Foerster, J., Tuyls, K., and Graepel, T. The mechanics of n-player differentiable games. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 354–363, 2018.

- Balduzzi, D., Garnelo, M., Bachrach, Y., Czarnecki, W., Perolat, J., Jaderberg, M., and Graepel, T. Open-ended learning in symmetric zero-sum games. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 434–443, 2019.
- Balogh, J., Békési, J., Dósa, G., Sgall, J., and Stee, R. v. The optimal absolute ratio for online bin packing. In *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 1425–1438. SIAM, 2014.
- Beasley, J. E. OR-Library: distributing test problems by electronic mail. *Journal of the Operational Research Society*, pp. 1069–1072, 1990.
- Bengio, Y., Lodi, A., and Prouvost, A. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, pp. 405–421, 2021.
- Bi, J., Ma, Y., Wang, J., Cao, Z., Chen, J., Sun, Y., and Chee, Y. M. Learning generalizable models for vehicle routing problems via knowledge distillation. *Advances in Neural Information Processing Systems*, pp. 31226–31238, 2022.
- Blum, C. and Roli, A. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys (CSUR)*, pp. 268–308, 2003.
- Brown, G. W. Iterative solution of games by fictitious play. In *Activity Analysis of Production and Allocation*, pp. 374–376. John Wiley & Sons, New York, 1951.
- Burke, E., Kendall, G., Newall, J., Hart, E., Ross, P., and Schulenburg, S. Hyper-heuristics: An emerging direction in modern search technology. In *Handbook of Metaheuristics*, pp. 457–474. 2003.
- Burke, E. K., Hyde, M. R., Kendall, G., Ochoa, G., Özcan, E., and Woodward, J. R. Exploring hyper-heuristic methodologies with genetic programming. In *Computational intelligence: Collaboration, fusion and emergence*, pp. 177–201. 2009.
- Burke, E. K., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., and Woodward, J. R. A classification of hyper-heuristic approaches. In Gendreau, M. and Potvin, J.-Y. (eds.), *Handbook of Metaheuristics*, pp. 449–468. Springer US, 2010a.
- Burke, E. K., Hyde, M., Kendall, G., and Woodward, J. A genetic programming hyper-heuristic approach for evolving 2-D strip packing heuristics. *IEEE Transactions on Evolutionary Computation*, pp. 942–958, 2010b.
- Burke, E. K., Gendreau, M., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., and Qu, R. Hyper-heuristics: A survey of the state of the art. *Journal of the Operational Research Society*, pp. 1695–1724, 2013.
- Burke, E. K., Hyde, M. R., Kendall, G., Ochoa, G., Özcan, E., and Woodward, J. R. A classification of hyper-heuristic approaches: revisited. In *Handbook of metaheuristics*, pp. 453–477. Springer, 2018.
- Castiñeiras, I., De Cauwer, M., and O’Sullivan, B. Weibull-based benchmarks for bin packing. In *International Conference on Principles and Practice of Constraint Programming*, pp. 207–222. Springer, 2012.
- Chen, J., Wang, J., Zhang, Z., Cao, Z., Ye, T., and Chen, S. Efficient meta neural heuristic for multi-objective combinatorial optimization. *Advances in Neural Information Processing Systems*, pp. 56825–56837, 2023.
- Chen, Y., Chen, R., Luo, F., and Wang, Z. Improving generalization of neural combinatorial optimization for vehicle routing problems via test-time projection learning. In *Advances in Neural Information Processing Systems*, 2025.
- Czarnecki, W. M., Gidel, G., Tracey, B., Tuyls, K., Omidshafiei, S., Balduzzi, D., and Jaderberg, M. Real world games look like spinning tops. *Advances in Neural Information Processing Systems*, pp. 17443–17454, 2020.
- Dat, P. V. T., Doan, L., and Binh, H. T. T. HSEvo: Elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 26931–26938, 2025.
- Delorme, M., Iori, M., and Martello, S. BPPLIB: a library for bin packing and cutting stock problems. *Optimization Letters*, 12(2):235–250, 2018.
- Dósa, G. and Sgall, J. First fit bin packing: A tight analysis. In *30th International Symposium on Theoretical Aspects of Computer Science (STACS 2013)*, pp. 538–549. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2013.
- Drake, J. H., Kheiri, A., Özcan, E., and Burke, E. K. Recent advances in selection hyper-heuristics. *European Journal of Operational Research*, 285(2):405–428, 2020.
- Duan, R., Liu, Y., Dong, X., and Fan, C. EALG: Evolutionary adversarial generation of language model-guided generators for combinatorial optimization. *arXiv preprint arXiv:2506.02594*, 2025.
- Duflo, G., Kieffer, E., Brust, M. R., Danoy, G., and Bouvry, P. A GP hyper-heuristic approach for generating TSP heuristics. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 521–529, 2019.
- Falkenauer, E. A hybrid grouping genetic algorithm for bin packing. *Journal of Heuristics*, 2(1):5–30, 1996.

- Freund, Y. and Schapire, R. E. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- Garey, M. R. and Johnson, D. S. Approximation algorithms for bin packing problems: A survey. In *Analysis and design of algorithms in combinatorial optimization*, pp. 147–172. Springer, 1981.
- Google OR-Tools Team. Google OR-Tools, 2024. URL <https://developers.google.com/optimization>.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Hemberg, E., Moskal, S., and O’Reilly, U.-M. Evolving code with a large language model. *Genetic Programming and Evolvable Machines*, 25(2):21, 2024.
- Hildebrandt, T., Heger, J., and Scholz-Reiter, B. Towards improved dispatching rules for complex shop floor scenarios: A genetic programming approach. In *Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation*, pp. 257–264. Association for Computing Machinery, 2010.
- Hutter, F., Hoos, H. H., and Leyton-Brown, K. Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization*, pp. 507–523. Springer, 2011.
- Iklassov, Z., Medvedev, D., De Retana, R. S. O., and Takac, M. On the study of curriculum learning for inferring dispatching policies on the job shop scheduling. In *International Joint Conference on Artificial Intelligence*, pp. 5350–5358, 2023.
- Jaderberg, M., Dalibard, V., Osindero, S., Czarnecki, W. M., Donahue, J., Razavi, A., Vinyals, O., Green, T., Dunning, I., Simonyan, K., et al. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.
- Jiang, C., Shu, X., Qian, H., Lu, X., Zhou, J., Zhou, A., and Yu, Y. LLMOPT: Learning to define and solve general optimization problems from scratch. In *International Conference on Learning Representations*, 2025.
- Jiang, Y., Wu, Y., Cao, Z., and Zhang, J. Learning to solve routing problems via distributionally robust optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 9786–9794, 2022.
- Johnson, D. S., Demers, A., Ullman, J. D., Garey, M. R., and Graham, R. L. Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal on Computing*, 3(4):299–325, 1974.
- Koza, J. R. Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, 4(2):87–112, 1994.
- Lanctot, M., Zambaldi, V., Gruslys, A., Lazaridou, A., Tuyls, K., Pérolat, J., Silver, D., and Graepel, T. A unified game-theoretic approach to multiagent reinforcement learning. *Advances in Neural Information Processing Systems*, 2017.
- Lisicki, M., Afkanpour, A., and Taylor, G. W. Evaluating curriculum learning strategies in neural combinatorial optimization. In *Learning Meets Combinatorial Algorithms at NeurIPS2020*, 2020.
- Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., et al. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025a.
- Liu, F., Lu, C., Gui, L., Zhang, Q., Tong, X., and Yuan, M. Heuristics for vehicle routing problem: A survey and recent advances. *arXiv preprint arXiv:2303.04147*, 2023a.
- Liu, F., Tong, X., Yuan, M., and Zhang, Q. Algorithm evolution using large language model. *arXiv preprint arXiv:2311.15249*, 2023b.
- Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., and Zhang, Q. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 32201–32223, 2024a.
- Liu, F., Liu, Y., Zhang, Q., Xialiang, T., and Yuan, M. EoH-S: Evolution of heuristic set using llms for automated heuristic design. *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 37090–37098, 2026.
- Liu, Y., Zhou, C., Zhang, P., Li, Z., Zhang, S., Lin, X., and Wu, X. CL4CO: A curriculum training framework for graph-based neural combinatorial optimization. In *2024 IEEE International Conference on Data Mining (ICDM)*, pp. 779–784, 2024b.
- Liu, Y., Li, J., Zhao, W. X., Lu, H., and Wen, J.-R. Experience-guided reflective co-evolution of prompts and heuristics for automatic algorithm design. *arXiv preprint arXiv:2509.24509*, 2025b.
- López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., and Stützle, T. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.

- Luo, F., Lin, X., Liu, F., Zhang, Q., and Wang, Z. Neural combinatorial optimization with heavy decoder: Toward large scale generalization. *Advances in Neural Information Processing Systems*, pp. 8845–8864, 2023.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhmoey, S., Yang, Y., et al. Self-Refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, pp. 46534–46594, 2023.
- Manchanda, S., Michel, S., Drakulic, D., and Andreoli, J.-M. On the generalization of neural combinatorial optimization heuristics. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pp. 426–442, 2022.
- Martello, S. and Toth, P. Lower bounds and reduction procedures for the bin packing problem. *Discrete applied mathematics*, 28(1):59–70, 1990.
- McMahan, H. B., Gordon, G. J., and Blum, A. Planning in the presence of cost functions controlled by an adversary. In *Proceedings of the 20th international conference on machine learning*, pp. 536–543, 2003.
- Muller, P., Omidshafiei, S., Rowland, M., Tuyls, K., Pérolat, J., Liu, S., Hennes, D., Marris, L., Lanctot, M., Hughes, E., Wang, Z., Lever, G., Heess, N., Graepel, T., and Munos, R. A generalized training approach for multiagent learning. In *International Conference on Learning Representations*, 2020.
- Novikov, A., Vű, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J., Mehrabian, A., et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Papadimitriou, C. H. and Steiglitz, K. *Combinatorial Optimization: Algorithms and Complexity*. Courier Corporation, 1998.
- Pillay, N. and Qu, R. *Hyper-heuristics: theory and applications*. Springer, 2018.
- Pluhacek, M., Kazikova, A., Kadavy, T., Viktorin, A., and Senkerik, R. Leveraging large language models for the generation of novel metaheuristic optimization algorithms. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, pp. 1812–1820, 2023.
- Qiu, R., Sun, Z., and Yang, Y. DIMES: A differentiable meta solver for combinatorial optimization problems. *Advances in Neural Information Processing Systems*, pp. 25531–25546, 2022.
- Ramanan, P., Brown, D. J., Lee, C.-C., and Lee, D.-T. On-line bin packing in linear time. *Journal of Algorithms*, 10(3):305–326, 1989.
- Reinelt, G. TSPLIB—A traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991.
- Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Rosenkrantz, D. J., Stearns, R. E., and Lewis, II, P. M. An analysis of several heuristics for the traveling salesman problem. *SIAM journal on computing*, 6(3):563–581, 1977.
- Seiden, S. S. On the online bin packing problem. *Journal of the ACM (JACM)*, 49(5):640–671, 2002.
- Shi, Y., Zhou, J., Song, W., Bi, J., Wu, Y., and Zhang, J. Generalizable heuristic generation through large language models with meta-optimization. In *International Conference on Learning Representations*, 2026.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, pp. 8634–8652, 2023.
- Shoham, Y. and Leyton-Brown, K. *Multiagent systems: Algorithmic, game-theoretic, and logical foundations*. Cambridge University Press, 2008.
- Sim, K., Renau, Q., and Hart, E. Beyond the hype: Benchmarking LLM-evolved heuristics for bin packing. In *International Conference on the Applications of Evolutionary Computation (Part of EvoStar)*, pp. 386–402, 2025.
- Son, J., Kim, M., Kim, H., and Park, J. Meta-SAGE: Scale meta-learning scheduled adaptation with guided exploration for mitigating scale shift on combinatorial optimization. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 32194–32210, 2023.
- Surina, A., Mansouri, A., Quaedy, L., Seddas, A., Vizovska, M., Abbe, E., and Gulcehre, C. Algorithm discovery with LLMs: Evolutionary search meets reinforcement learning. In *Second Conference on Language Modeling*, 2025.
- Timbers, F., Bard, N., Lockhart, E., Lanctot, M., Schmid, M., Burch, N., Schrittwieser, J., Hubert, T., and Bowling, M. Approximate exploitability: Learning a best response. In *International Joint Conference on Artificial Intelligence*, pp. 3487–3493, 2022.

- Toth, P. and Vigo, D. *Vehicle routing: problems, methods, and applications*. SIAM, 2014.
- Uchoa, E., Pecin, D., Pessoa, A., Poggi, M., Vidal, T., and Subramanian, A. New benchmark instances for the capacitated vehicle routing problem. *European Journal of Operational Research*, 257(3):845–858, 2017.
- Ullman, J. The performance of a memory allocation algorithm. 1971.
- van Stein, N. and Bäck, T. LLaMEA: A large language model evolutionary algorithm for automatically generating metaheuristics. *IEEE Transactions on Evolutionary Computation*, 2024.
- Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575 (7782):350–354, 2019.
- von Neumann, J. On the theory of games of strategy. *Contributions to the Theory of Games*, 1959.
- Voudouris, C. and Tsang, E. Guided local search and its application to the traveling salesman problem. *European Journal of Operational Research*, 113(2):469–499, 1999.
- Wang, C., Yang, Y., Slumbers, O., Han, C., Guo, T., Zhang, H., and Wang, J. A game-theoretic approach for improving generalization ability of TSP solvers. In *ICLR 2022 Workshop on Gamification and Multiagent Solutions*, 2022.
- Wang, C., Yu, Z., McAleer, S., Yu, T., and Yang, Y. ASP: Learn a universal neural solver! *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(6):4102–4114, 2024.
- Wang, H. P. and Li, P. Unsupervised learning for combinatorial optimization needs meta learning. In *International Conference on Learning Representations*, 2023.
- Wellman, M. P. Methods for empirical game-theoretic analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 1552–1556, 2006.
- Wouda, N. A., Lan, L., and Kool, W. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 36(4):943–955, 2024.
- Yao, S., Liu, F., Lin, X., Lu, Z., Wang, Z., and Zhang, Q. Multi-objective evolution of heuristic using large language model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 27144–27152, 2025.
- Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., and Song, G. ReEvo: Large language models as hyper-heuristics with reflective evolution. *Advances in Neural Information Processing Systems*, pp. 43571–43608, 2024.
- Zhang, D., Xiao, Z., Wang, Y., Song, M., and Chen, G. Neural TSP solver with progressive distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 12147–12154, 2023.
- Zhang, Z., Zhang, Z., Wang, X., and Zhu, W. Learning to solve travelling salesman problem with hardness-adaptive curriculum. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 9136–9144, 2022.
- Zheng, Y., Luo, F., Wang, Z., Wu, Y., and Zhou, Y. MTL-KD: Multi-task learning via knowledge distillation for generalizable neural vehicle routing solver. In *Advances in Neural Information Processing Systems*, pp. 95546–95570, 2025.
- Zhou, J., Wu, Y., Song, W., Cao, Z., and Zhang, J. Towards omni-generalizable neural methods for vehicle routing problems. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 42769–42789, 2023.

A. Policy-Space Response Oracle

A.1. Game-Theoretic Setup

We consider a general N -player normal-form game. Each player $i \in \{1, \dots, N\}$ chooses a strategy π_i from a (possibly infinite) strategy space Π_i . The joint strategy profile is denoted by $\pi = (\pi_1, \dots, \pi_N)$, and π_{-i} refers to the strategies of all players except i . Player i receives utility $u_i(\pi_i, \pi_{-i}) \in \mathbb{R}$.

A.2. Restricted Games and Meta-Strategies

Directly solving games over the full strategy spaces Π_i is generally intractable. PSRO (Lanctot et al., 2017) addresses this by iteratively constructing a *restricted game*. At iteration t , each player i maintains a finite set of candidate strategies $\Pi_i^{(t)} \subset \Pi_i$. The restricted game is defined by evaluating all joint strategy profiles drawn from $\Pi_1^{(t)} \times \dots \times \Pi_N^{(t)}$. A *meta-strategy* $\sigma_i^{(t)}$ is a mixed strategy over $\Pi_i^{(t)}$, i.e., a probability distribution satisfying

$$\sigma_i^{(t)} \in \Delta(\Pi_i^{(t)}).$$

Here, $\Delta(\Pi_i^{(t)})$ denotes the probability simplex over the finite set $\Pi_i^{(t)}$. In practical implementations, a meta-strategy often samples a strategy $\pi_i \sim \sigma_i^{(t)}$ and uses it for evaluation within that iteration.

When the restricted game is finite, computing meta-strategies reduces to solving a standard equilibrium problem over a payoff matrix. For clarity, we illustrate meta-strategy computation in a two-player zero-sum setting. In this case, the game is fully characterized by the utility of player 1, with $u_2(\pi_1, \pi_2) = -u_1(\pi_1, \pi_2)$. The restricted game can therefore be represented by a payoff matrix $M^{(t)} \in \mathbb{R}^{m \times n}$ with entries

$$M_{ij}^{(t)} = u_1(\pi_1^i, \pi_2^j),$$

where $\pi_1^i \in \Pi_1^{(t)}$ and $\pi_2^j \in \Pi_2^{(t)}$.

The meta-strategy for player 1 can then be obtained by solving the following linear program, as implied by the minimax theorem for two-player zero-sum games (von Neumann, 1959; Shoham & Leyton-Brown, 2008):

$$\begin{aligned} \max_{x, v} \quad & v \\ \text{s.t.} \quad & x^\top M^{(t)} \geq v \mathbf{1}, \\ & x \geq 0, \quad \mathbf{1}^\top x = 1, \end{aligned}$$

where x corresponds to the mixed strategy $\sigma_1^{(t)}$ over $\Pi_1^{(t)}$. An analogous linear program yields the meta-strategy for player 2.

A.3. Best-Response Oracles

Given opponents' meta-strategies $\sigma_{-i}^{(t)}$, a best-response (BR) for player i is defined as a strategy that maximizes expected utility against the induced distribution over opponent strategies:

$$\pi_{i, \text{BR}} \in \arg \max_{\pi \in \Pi_i} \mathbb{E}_{\pi_{-i} \sim \sigma_{-i}^{(t)}} [u_i(\pi, \pi_{-i})].$$

In practice, computing exact best responses is rarely tractable. PSRO therefore relies on approximate BR oracles (Lanctot et al., 2017), whose role is to return a high-performing strategy under the current opponent meta-strategy, rather than a provably optimal one. The above definition assumes utility maximization; minimization objectives can be handled by negating the payoff without loss of generality.

A.4. PSRO Algorithm

PSRO proceeds in an iterative “solve-and-expand” loop:

1. **Initialization.** Initialize each population $\Pi_i^{(0)}$ with a small set of strategies.
2. **Meta-game solving.** At iteration t , construct the restricted game induced by $\{\Pi_i^{(t)}\}_{i=1}^N$ and compute a meta-strategy $\sigma^{(t)} = (\sigma_1^{(t)}, \dots, \sigma_N^{(t)})$ using a suitable equilibrium solver.
3. **Response-based expansion.** For one or more players, invoke a BR oracle against $\sigma_{-i}^{(t)}$ to obtain a new strategy, which is added to the population:

$$\Pi_i^{(t+1)} \leftarrow \Pi_i^{(t)} \cup \{\pi_{i,\text{new}}\}.$$

By iteratively expanding the restricted game with targeted best responses, PSRO refines the approximation of the underlying full game.

B. EoH-style Program Search Mechanism

Within ASRO, we instantiate the program-space best-response oracle using an EoH-style evolutionary program search mechanism (Liu et al., 2024a). Conditioned on the opponent’s meta-strategy $\sigma_{-i}^{(t)}$, this mechanism evolves a population of solver or generator programs via LLM-driven variation and selection to approximately optimize the induced best-response objective.

In this mechanism, each program is represented through a natural-language “thought” that guides LLM reasoning and an executable “code” component that is directly evaluated. This dual representation enables both conceptual and syntactic variation during evolution in practice.

B.1. Evolutionary Operators and Prompt Strategies

The EoH-style search mechanism includes one initialization operator and five variation operators, grouped into exploration and modification categories to generate diverse and progressively refined heuristics.

Initialization (I1). At the first ASRO iteration, I1 constructs the initial population by prompting the LLM to produce heuristics entirely from scratch. This provides a starting population for subsequent evolution.

Exploration and modification operators follow a unified usage pattern: in each evolutionary round, a parent set of size p is sampled from the current population; an operator-specific prompt is applied; and the LLM outputs a new heuristic derived from these parents.

Exploration Operator E1 (Diverse Exploration). E1 encourages large jumps in program space by asking the LLM to design a heuristic that differs substantially from all selected parents. Its role is to introduce large, conceptually novel variations that sustain population diversity.

Exploration Operator E2 (Shared-Idea Recombination). E2 performs semantic recombination. The LLM is instructed to identify the common idea shared by the parents and then construct a new heuristic that preserves this idea while introducing new structural elements. This operator encourages abstraction and meaningful recombination of successful patterns.

Modification Operator M1 (Semantic Refinement). M1 improves a single parent by refining its underlying idea. The LLM analyzes the heuristic’s logic and proposes targeted adjustments that strengthen its conceptual soundness or operational behavior.

Modification Operator M2 (Parameterized Adjustment). M2 performs local numerical tuning. It directs the LLM to adjust constants, thresholds, or coefficients without altering the heuristic’s overall structure, enabling fine-grained performance improvements.

Modification Operator M3 (Simplification and Pruning). M3 simplifies a parent heuristic by identifying and removing redundant or overly complex components while preserving the core idea. This promotes parsimony and often yields heuristics with stronger generalization.

B.2. Mechanism Workflow and Iterative Search Process

The EoH-style search mechanism performs program-space search through an iterative evolutionary process that combines LLM-generated program variations with fitness-based population updates. Algorithm 2 provides the procedural view of a

EoH Prompt Template Structure

1. Task Description

Defines the optimization problem and the required I/O format.

2. Operator-Specific Prompt

Instruction for the chosen operator (I1, E1, E2, M1, M2, M3). Specifies how the LLM should reason over parent heuristics and produce a variant.

3. Expected Output

LLM must first describe the heuristic idea, then provide a full Python function with a fixed signature (name, inputs, outputs).

4. Note

Additional constraints: determinism, no extra imports, concise response, and strict adherence to the required code format.

5. Parent Heuristics

Includes the thought and code of p parent heuristics (empty only in I1). Serves as in-context examples for reasoning and variation.

Figure 5. Unified structure of all prompt templates used by EoH evolutionary operators. Each operator (I1, E1, E2, M1, M2, M3) uses this five-part template with different operator-specific instructions.

single best-response search, while this section summarizes the conceptual workflow. Here, p , K , and R denote user-defined hyperparameters controlling parent selection, population size, and evolutionary depth, respectively.

At the beginning of iteration t in ASRO, the search mechanism initializes a population $\mathcal{P}_0$ of candidate programs. If $t = 0$, the population is created via prompt-based synthesis; otherwise, it is inherited from the previous ASRO iteration. Each program is represented by a thought-code pair and evaluated against the fixed opponent meta-strategy $\sigma_{-i}^{(t)}$.

The mechanism then runs R evolutionary rounds, where R is a user-defined parameter controlling the search depth. In each round, a subset of parents $\mathcal{P}_{\text{sel}}$ is selected according to fitness and diversity. LLM operators (I1, E1, E2, M1, M2, M3) are instantiated as prompt templates (Figure 5), and each operator generates new program candidates by transforming the selected parents. These candidates are evaluated under $\sigma_{-i}^{(t)}$, merged with the existing population, and truncated to maintain a fixed size K . After R rounds, the highest-performing program in $\mathcal{P}_R$ is returned as the approximate best response.

This iterative scheme enables the search mechanism to explore both large conceptual variations (via exploration operators) and fine-grained improvements (via modification operators), while maintaining diversity across evolution. The parameter R trades optimization depth for computational cost: larger R allows richer program refinement but increases total LLM calls. In all experiments, R is kept moderate to balance performance and efficiency.

B.3. Evaluation under Mixed Strategies

At each ASRO iteration, candidate programs are evaluated as approximate best responses to a fixed opponent meta-strategy $\sigma_{-i}^{(t)}$. This requires estimating expected performance under a mixed strategy rather than against a single opponent.

Solver Evaluation. When evolving solver programs, the opponent strategy is the generator mixture $\sigma_g^{(t)}$. For a solver candidate s , evaluation proceeds by first considering each generator g_j in the current population. A fixed set of n_I instances is sampled from g_j , and the solver is applied to all sampled instances. Here, n_I denotes the number of problem instances sampled per generator for evaluation. Gaps are averaged within each generator to obtain a per-generator estimate

$$\widehat{\text{gap}}(s, g_j) = \frac{1}{n_I} \sum_{k=1}^{n_I} \text{gap}(s, x_{j,k}), \quad x_{j,k} \sim g_j.$$

where gap denotes the normalized reference gap defined in Eq. 2. The overall performance of s under the mixed strategy is then computed by aggregating across generators according to $\sigma_g^{(t)}$:

Algorithm 2 Program-Space Best-Response Search (LLM-Based)

```

1: Input: Opponent meta-strategy  $\sigma_{-i}$ ; LLM operators  $\mathcal{O}$ ; population size  $K$ ; iterations  $R$ 
2: Output: Best-performing program  $p^*$ 
3: (0) Initialization
4: if first iteration then
5:    $\mathcal{P}^{(0)} \leftarrow$  Prompt-based synthesis
6: else
7:    $\mathcal{P}^{(0)} \leftarrow$  Inherit from previous population
8: end if
9: for  $r = 1$  to  $R$  do
10:  (1) Selection
11:  Select parent set  $\mathcal{P}_{\text{sel}} \subset \mathcal{P}^{(r-1)}$  based on fitness
12:  (2) LLM-Based Variation
13:   $\mathcal{C}^{(r)} \leftarrow \{\text{LLM}(p, o) \mid p \in \mathcal{P}_{\text{sel}}, o \in \mathcal{O}\}$ 
14:  (3) Evaluation Against Meta-Strategy
15:  Evaluate all  $p \in \mathcal{C}^{(r)} \cup \mathcal{P}^{(r-1)}$  against  $\sigma_{-i}$ 
16:  (4) Population Update
17:   $\mathcal{P}^{(r)} \leftarrow \text{TOP-}K(\mathcal{C}^{(r)} \cup \mathcal{P}^{(r-1)})$ 
18: end for
19:  $p^* \leftarrow \arg \max_{p \in \mathcal{P}^{(R)}} f(p)$ 
    
```

$$\widehat{\text{gap}}(s \mid \sigma_g^{(t)}) = \sum_j \sigma_g^{(t)}(g_j) \widehat{\text{gap}}(s, g_j).$$

Generator Evaluation. Generator candidates are evaluated analogously under the solver mixture $\sigma_s^{(t)}$. For a generator g , all solver programs s_ℓ are applied to instances sampled from g . Gaps are first averaged within each solver,

$$\widehat{\text{gap}}(s_\ell, g) = \frac{1}{n_I} \sum_{k=1}^{n_I} \text{gap}(s_\ell, x_k),$$

and final performance is obtained by aggregating across solvers according to $\sigma_s^{(t)}$:

$$\widehat{\text{gap}}(g \mid \sigma_s^{(t)}) = \sum_\ell \sigma_s^{(t)}(s_\ell) \widehat{\text{gap}}(s_\ell, g).$$

For generators, higher $\widehat{\text{gap}}(g \mid \sigma_s^{(t)})$ indicates a stronger adversary and is therefore preferred. In both cases, evaluation reduces to computing weighted empirical averages over solver–generator–instance interactions, where the weights are given by the opponent meta-strategy.

C. Batch Evaluation Engine

At each meta-game iteration of ASRO, both best-response search and payoff matrix construction require evaluating a large number of solver–generator interactions under fixed opponent mixed strategies, as defined in Section B.3. These evaluations correspond to computing the mixed-strategy expectations introduced above.

Concretely, given n_s solver programs, n_g generator programs, and n_I instances sampled per generator, a single meta-game iteration involves

$$O(n_s \times n_g \times n_I)$$

solver executions.

Table 8. Hyperparameter settings used in ASRO experiments across domains.

Task	K	R_s	R_g	n_I	min-ratio
OBP	20	4	2	2	0.3
TSP	10	2	2	3	0.4
CVRP	10	2	2	5	0.3

To implement this evaluation workload in a unified and computationally efficient manner, we adopt a batch evaluation engine that materializes all required evaluation tasks and executes them in parallel.

C.1. Unified Two-Stage Evaluation Pipeline

The batch evaluation engine follows a unified two-stage pipeline consisting of task preparation and batch execution.

Stage I: Task Preparation. The preparation stage constructs the complete set of evaluation tasks corresponding to solver-generator-instance triples. First, each generator samples a fixed set of n_I problem instances. For a given generator, the same instance set is shared across all solvers to ensure consistent comparison. If required by the gap definition, reference values (e.g., optimal solutions or bounds) are computed for all generated instances. All solver programs are then pre-processed (e.g., compilation or validation) once prior to execution, avoiding redundant overhead during evaluation and isolating failures of individual solver programs. Finally, evaluation tasks are constructed by enumerating all combinations. Each task encapsulates the solver identity, generator identity, instance data, the reference value (if applicable), and the corresponding opponent meta-strategy weight.

Stage II: Batch Task Execution. All prepared tasks are executed in batch. Each task corresponds to one solver execution on a single instance and returns the observed normalized gap. Results are grouped by solver-generator pairs and averaged across instances to produce empirical estimates of meta-game payoffs.

C.2. Parallelization and Robustness

Parallelism is applied internally within both preparation and execution stages to accelerate evaluation at scale. Independent resource budgets are used to prevent CPU oversubscription, and all evaluation tasks are protected by timeouts. Failures of individual solver executions are isolated and penalized without interrupting the overall evaluation process.

D. Additional Experimental Details

Hyperparameters. Table 8 summarizes the key hyperparameters used in ASRO across domains. The outer iteration count T specifies the number of solver-generator meta-game expansions. At each iteration, fixed-size solver and generator populations of size K are maintained. The program-space best-response oracle is implemented via an EoH-style evolutionary search, where R_s and R_g denote the numbers of evolutionary rounds for synthesizing solver and generator candidates, respectively. Each candidate program is evaluated on n_I instances sampled from the corresponding generator program. To stabilize early-stage co-evolution, a fixed fraction of instances (min-ratio) is always drawn from a task-specific base generator.

In terms of LLM usage, each best-response oracle invocation synthesizes $R \times K$ candidate programs *for each* EoH operator in a fixed operator set, with an additional one-time initialization at the first ASRO iteration; this procedure is applied symmetrically to both solver and generator at every iteration.

All experiments are conducted on a single-node server equipped with Intel Xeon Platinum 8280 CPUs (4 sockets $\times$ 28 cores $\times$ 2 threads, 224 logical CPUs). Program evaluations are parallelized using up to 128 process-based workers via joblib, where independent (solver, instance) pairs are evaluated concurrently. Intermediate evaluation results are cached within each iteration to avoid redundant computation.

E. Online Bin Packing (OBP)

Problem Definition. We consider the *online bin packing problem*, where items arrive sequentially and must be assigned irrevocably to bins of fixed capacity C (Seiden, 2002). At each time step t , an item of size $w_t \in (0, C]$ arrives and must be placed into an existing bin with sufficient remaining capacity or into a newly opened bin. Once placed, items cannot be

Figure 6. Base prompt examples for solver and generator synthesis in online bin packing (OBP).

Prompt for Generator Task. Write a novel Python function to generate sequences of items for the online bin packing problem. Goal: Create instances that are as difficult as possible for heuristic solvers to pack efficiently. The difficulty must arise from the item arrival order and the strategic relationship between sequential items. I/O. Implement <code>generate_instances(seeds, capacity, num_items)</code> and return instances. Input/Output Info. seeds: iterable of integers, one per instance. capacity: bin capacity (int > 1). num_items: number of items in each instance (int > 0). instances: Python list with the same length as seeds; each element is a NumPy integer array named <code>items</code> of shape <code>(num_items,)</code> with values strictly in <code>[1, capacity - 1]</code>. Other Info. Avoid simple random noise; instead, design structured sequences that force poor decision-making in real-time assignments. Avoid trivial or degenerate streams that make the objective uninformative. Output ONLY the function code.	Prompt for Solver Task. Write a novel scoring Python function for the online bin packing problem using a greedy strategy. Goal: Minimize the total number of bins by making each assignment facilitate future packing. Strategy Hint: Consider not just the tightest fit, but the potential usability of the remaining space. A good score should balance between filling a bin and avoiding leaving behind tiny, unusable gaps that might reject future items. I/O. Implement <code>score(item, bins)</code> and return scores. Input/Output Info. item: current item size. bins: NumPy array of remaining capacities for <i>feasible bins only</i> (each entry satisfies <code>bins[i] >= item</code>). scores: NumPy array with the same shape as bins. Higher is better. Other Info. Be scale-invariant (use ratios such as <code>(bins - item)/max(bins)</code> or statistics of bins instead of hard-coded thresholds). Output ONLY the function code.
--	---

moved or reassigned. The objective is to minimize the total number of bins used.

An instance is defined by an item sequence

$$x = (w_1, w_2, \dots, w_T),$$

and the solver must make decisions online, without lookahead or reordering.

E.1. Solver Program Interface.

Each solver program defines a numerical priority function

$$\text{score}(w, \mathbf{r}),$$

which takes the current item size w and a vector $\mathbf{r}$ of remaining capacities of all feasible bins, and returns a score vector aligned with $\mathbf{r}$ (one score per feasible bin). Only the priority function varies across solvers; the decoding logic and online constraints are fixed.

E.2. Prompt Configuration

We first specify task-level base prompts for online bin packing, as illustrated in Figure 6. These prompts define the OBP task, including the task description, the required function interfaces, and task-specific constraints for solver and generator synthesis.

During evolution, these task-level prompts are embedded into a unified prompt template (Figure 5). Specifically, the *Task* section of the OBP prompt is placed in the *Task Description* slot, defining the optimization objective and problem setting. The function signatures specified under *I/O* are enforced in the *Expected Output* slot. Detailed input/output specifications and additional constraints provided under *Input/Output Info* and *Other Info* are incorporated into the *Note* section. All remaining components of the prompt, including operator-specific instructions and the inclusion of parent programs, follow the standard template of the search mechanism without modification.

E.3. Evolution Outcomes

Figure 7 provides a qualitative snapshot of the solver–generator strategy space shaped by co-evolution in ASRO. The figure visualizes representative executable programs selected from the final solver and generator pools, highlighting characteristic

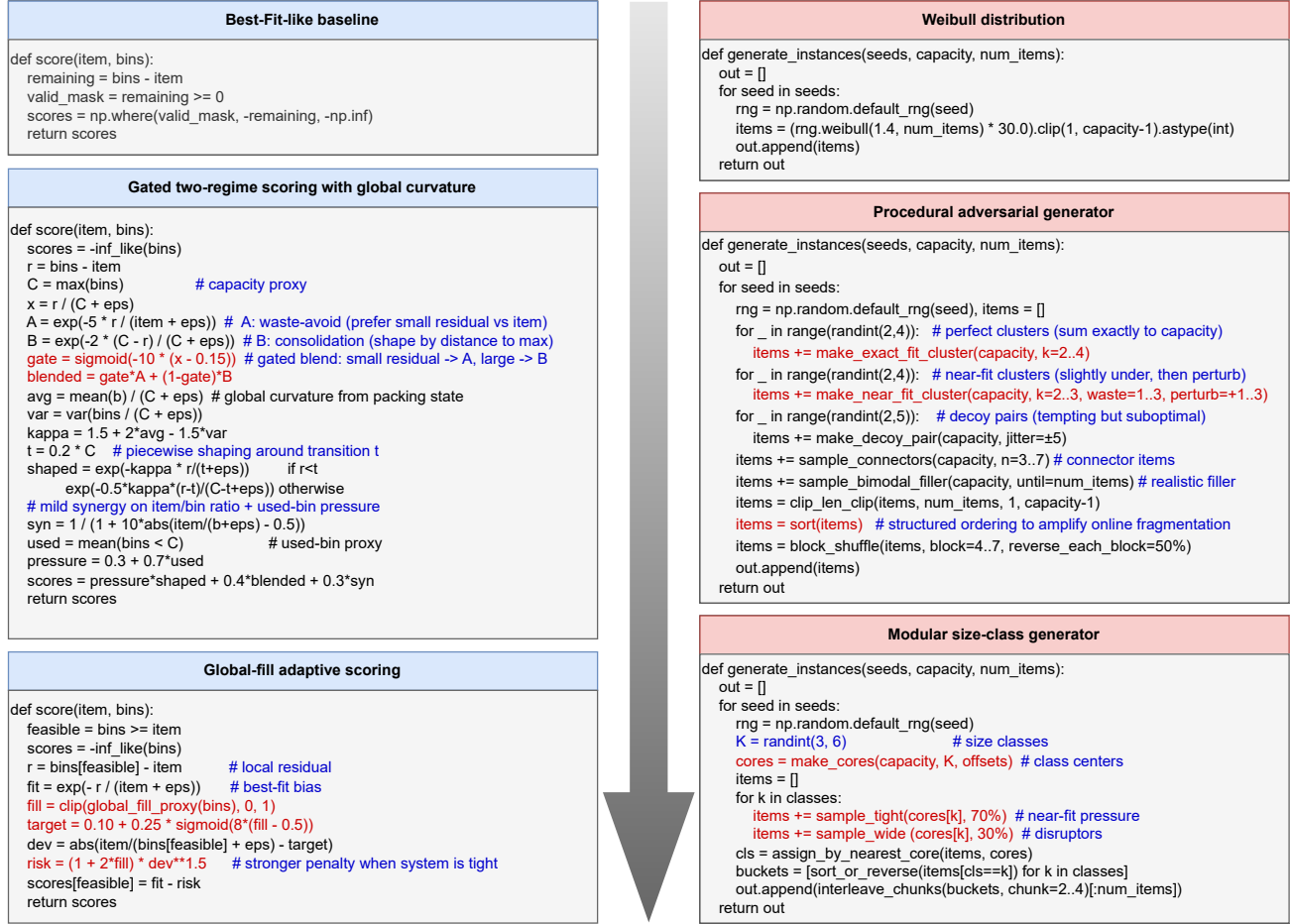


Figure 7. Representative solver and generator programs discovered by ASRO on online bin packing. The figure shows representative solver scoring rules (left) and instance generators (right) selected from the final solver and generator pools. Code is schematized to emphasize algorithmic structure rather than implementation details.

structural patterns induced by sustained solver–generator interaction. On the solver side, co-evolution with adversarial instance generators gives rise to scoring rules that extend beyond purely local heuristics by incorporating global packing statistics and adaptive penalty mechanisms. On the generator side, pressure from increasingly capable solvers leads to structured instance constructions that induce systematic pairing conflicts and amplify online misallocation. Together, these examples illustrate how solver and generator strategies mutually adapt under ASRO, resulting in a diverse and interdependent strategy space.

E.4. Datasets and Arrival Orders.

We evaluate OBP solvers on a collection of widely used public benchmarks as well as synthetic distributions. Specifically, we use the Falkenauer T and U benchmark families (Falkenauer, 1996), the Hard28 benchmark (Delorme et al., 2018). We additionally evaluate on synthetic instances generated from Weibull distributions, as commonly used in online bin packing studies (Castiñeiras et al., 2012). These datasets cover both classical test cases and structurally challenging scenarios commonly studied in the online bin packing literature.

Since online bin packing performance is sensitive to item arrival order (Albers et al., 2021), we evaluate all methods under multiple arrival models. In particular, we consider random order and size-based orders (ascending). Arrival orders are fixed independently of the evaluated algorithms and applied consistently across all methods to ensure fair comparison.

E.5. Baselines.

We compare ASRO-generated solvers against classical online bin packing heuristics and the EoH baseline. The classical baselines include *First Fit* (Ullman, 1971; Dósa & Sgall, 2013) and *Best Fit* (Johnson et al., 1974; Albers et al., 2021), which represent standard greedy strategies widely used in practice. In addition, we include the solver evolved by *Evolution of Heuristics (EoH)* (Liu et al., 2024a), which follows a static, solver-centric LLM-based heuristic discovery pipeline without adversarial instance generation. All baselines operate under the same online constraints and decoding procedure as ASRO solvers, without lookahead, reordering, or post-processing.

F. Traveling Salesman Problem (TSP)

F.1. Problem Definition.

We consider the Euclidean Traveling Salesman Problem (TSP) with N cities. Given city coordinates $\{c_i\}_{i=1}^N \subset \mathbb{R}^2$, the objective is to find a Hamiltonian tour of minimum total length. Equivalently, for a permutation π of $\{1, \dots, N\}$, the tour cost is

$$V(\pi) = \sum_{i=1}^N d(c_{\pi_i}, c_{\pi_{i+1}}), \quad \text{with } \pi_{N+1} = \pi_1,$$

where $d(\cdot, \cdot)$ is the Euclidean distance. The solver aims to minimize $V(\pi)$.

F.2. Solver Procedure and Program Interface (TSP)

Solver Procedure (Guided Local Search). For the Traveling Salesman Problem (TSP), we adopt Guided Local Search (GLS) (Voudouris & Tsang, 1999; Alsheddy et al., 2016; Arnold & Sörensen, 2019) as the fixed solver procedure. GLS is a local-search-based algorithm that iteratively improves a candidate tour using a predefined set of neighborhood operators, specifically 2-opt and relocate. At any point in the search, candidate local moves are evaluated under an *augmented edge-distance matrix* $D = (d_{ij})$, which is initialized from the Euclidean distances and subsequently modified by solver-defined update rules to incorporate adaptive edge penalties.

Starting from an initial tour, GLS repeatedly applies local search operators under D until a locally optimal tour π^{loc} is reached, i.e., no improving local move exists under D . Based on π^{loc} , GLS updates an edge-usage matrix $U = (u_{ij})$, where u_{ij} records how frequently edge (i, j) has been selected in locally optimal tours during the search history. The search then continues by updating the augmented edge-distance matrix via the solver-defined update rule and resuming local search under the updated distances.

Solver Program Interface. Within this fixed GLS procedure, solver programs specify an *edge-distance update rule* that modifies the augmented edge-distance matrix. Given the current augmented edge-distance matrix D , the current locally optimal tour π^{loc} , and the edge-usage matrix U , the solver program outputs an updated augmented edge-distance matrix:

$$\tilde{D} = \text{update_edge_distance}(D, \pi^{\text{loc}}, U).$$

The updated matrix $\tilde{D}$ is then used by GLS to evaluate subsequent local moves. All other components of the solver procedure, including the local search operators and the GLS control logic, are fixed across methods; only the distance-update rule varies across solver programs.

F.3. Prompt Configuration

We specify task-level base prompts for the traveling salesman problem, as illustrated in Figure 8. These prompts define the TSP task, including the optimization objective, required function interfaces, and task-specific constraints for solver and generator synthesis. Prompt construction for TSP follows the same procedure as in online bin packing. Specifically, the TSP base prompts are embedded into the unified prompt template structure shown in Figure 5, with task-specific content populated accordingly and all remaining components following the standard template of the search mechanism.

Figure 8. Base prompt examples for solver and generator synthesis in Traveling Salesman Problem (TSP).

Prompt for Generator Task. Write a novel Python function to generate Euclidean Traveling Salesman Problem instances. The goal is to create instances that are as difficult as possible for heuristic solvers, measured by the resulting tour length. The difficulty should arise from the geometric structure of city coordinates and their spatial arrangement, rather than from simple random noise. Design instances that induce misleading local structures, long-range dependencies, or deceptive spatial patterns that make greedy or local-search-based solvers struggle. I/O. Implement <code>generate_instances(seeds, n_cities)</code> and return instances. Input/Output Info. <code>seeds</code>: iterable of integers, one per instance. <code>n_cities</code>: number of cities in each TSP instance. <code>instances</code>: Python list with the same length as <code>seeds</code>; each element is a NumPy array of shape $(n_cities, 2)$, representing city coordinates in $[0, 1]^2$. Other Info. Output ONLY the function code.	Prompt for Solver Task. Write a GLS-style edge-distance update rule as a Python function that helps the solver minimize total tour length. The update rule should modify edge penalties based on the structure of a locally optimized tour, encouraging exploration of alternative edges and discouraging repeatedly used suboptimal edges. The goal is to improve solution quality across the sampled instance distribution, not just on a single instance. I/O. Implement <code>update_edge_distance(edge_distance, local_opt_tour, edge_n_used)</code> and return <code>updated_edge_distance</code>. Input/Output Info. <code>edge_distance</code>: NumPy array of shape (N, N), symmetric. <code>local_opt_tour</code>: list or 1D NumPy array of city indices representing a tour. <code>edge_n_used</code>: NumPy array of shape (N, N), symmetric, storing edge usage counts. <code>updated_edge_distance</code>: NumPy array of shape (N, N), symmetric. Other Info. Do NOT modify input arrays in-place; operate on copies and return updated copies. Output ONLY the function code.
---	---

F.4. Evolution Outcomes

Figure 9 shows representative solver and generator programs learned by ASRO on the Traveling Salesman Problem. Solver programs evolve toward more structured distance-update mechanisms, while instance generators progressively induce structured geometries that interact with these updates.

F.5. Benchmark Datasets

We evaluate the learned TSP solvers on benchmark instances from *TSPLIB* (Reinelt, 1991), a standard library of Traveling Salesman Problem instances widely used in the combinatorial optimization literature. *TSPLIB* instances exhibit substantial diversity in both scale and geometric structure, including uniform and clustered point sets, grid-like layouts, and instances with irregular spatial patterns.

The benchmark covers a broad range of problem sizes, from small instances with fewer than 200 cities to large-scale instances exceeding 1,000 cities. All *TSPLIB* instances considered are evaluated against reference tour lengths (e.g., best-known solutions), enabling standardized reporting via normalized optimality gaps.

Importantly, *TSPLIB* instances are not drawn from the distributions used during training. Evaluating on *TSPLIB* therefore provides a stringent test of generalization under distributional and structural shift.

F.6. More Results: Per-instance TSPLIB Performance

Table 9 reports per-instance results on *TSPLIB* benchmarks, comparing the final ASRO solver against the EoH baseline. Rather than aggregating by instance size or family, this table exposes instance-level behavior and highlights where improvements are consistently realized.

F.7. Baselines

Classical constructive heuristics include Nearest Insertion (NI) and Farthest Insertion (FI) (Applegate et al., 2011; Rosenkrantz et al., 1977), which build tours incrementally according to distance-based criteria. These methods are

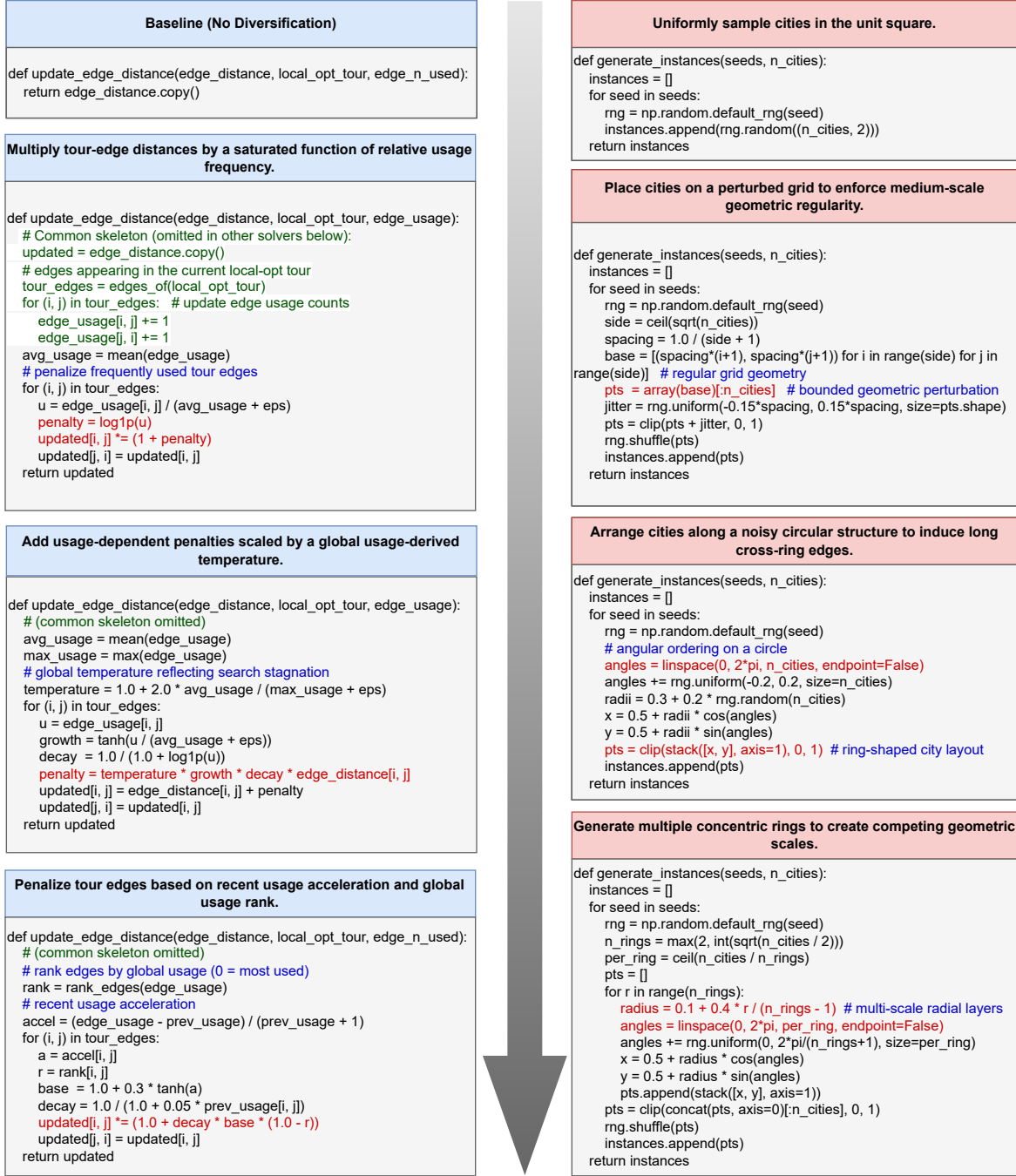


Figure 9. Representative solver and generator programs discovered by ASRO on the Traveling Salesman Problem. We show distilled solver distance-update rules (left) and instance generators (right) drawn from the final solver and generator pools. Code is schematized for clarity to emphasize algorithmic structure; auxiliary implementation details are omitted.

simple, deterministic, and widely used as standard non-learning baselines.

G. Capacitated Vehicle Routing Problem (CVRP)

G.1. Problem Definition.

We consider the Capacitated Vehicle Routing Problem (CVRP) (Toth & Vigo, 2014). A fleet of identical vehicles departs from and returns to a single depot to serve a set of customers with known demands. Each customer must be visited exactly

Table 9. Per-instance TSPLIB results. Lower is better.

Instance	ASRO-EoH	EoH	Instance	ASRO-EoH	EoH	Instance	ASRO-EoH	EoH	Instance	ASRO-EoH	EoH
A280	0.301	2.195	FNL4461	4.635	4.635	PCB442	0.462	0.990	RD400	0.163	2.717
BERLIN52	0.031	2.279	GIL262	0.414	0.955	PR1002	2.245	3.051	RL1304	3.780	6.771
BIER127	0.318	0.224	KROA100	0.016	0.119	PR107	-0.003	0.614	RL1323	2.582	4.764
CH130	0.012	0.381	KROA150	0.003	0.639	PR124	0.001	0.096	RL1889	2.784	3.879
CH150	0.044	1.282	KROA200	0.005	0.907	PR136	-0.001	0.459	ST70	0.313	0.313
D1291	3.649	5.484	KROB100	-0.009	-0.009	PR144	-0.003	0.353	TS225	2.582	4.764
D1655	4.006	4.144	KROB150	-0.009	1.367	PR152	0.002	0.907	TSP225	-0.901	0.561
D198	0.301	1.056	KROB200	0.015	0.819	PR226	0.002	0.877	U1060	1.850	4.132
D2103	1.068	1.839	KROC100	0.008	0.008	PR2392	4.186	4.186	U1432	2.645	4.069
D493	1.249	2.998	KROD100	0.001	0.317	PR264	0.000	0.231	U159	-0.010	-0.010
D657	1.012	3.023	KROE100	0.024	0.438	PR299	0.039	0.791	U1817	4.072	4.168
EIL101	1.782	4.144	LIN105	0.028	0.028	PR439	2.255	5.300	U2152	4.025	4.491
EIL51	0.674	0.700	LIN318	0.342	3.738	PR76	0.000	1.467	U2319	2.366	2.366
EIL76	1.184	1.732	LINHP318	2.002	5.454	RAT195	0.582	0.982	U574	1.174	5.071
FL1400	1.824	5.096	NRW1379	2.829	3.961	RAT575	2.183	3.152	U724	1.411	3.161
FL1577	2.409	5.031	P654	0.404	2.080	RAT783	2.363	3.907	VM1084	1.158	3.552
FL3795	4.382	4.382	PCB1173	3.246	5.004	RAT99	0.681	0.681	VM1748	2.133	3.280
FL417	0.509	0.984	PCB3038	4.132	4.132	RD100	0.005	0.016			

once, and the total demand served by any vehicle cannot exceed the vehicle capacity Q . The objective is to minimize the total travel cost over all routes. An instance is defined by a depot location, customer locations $\{c_i\}_{i=1}^N$, customer demands $\{d_i\}_{i=1}^N$, and a vehicle capacity Q . Feasibility with respect to capacity constraints must be satisfied at all times.

G.2. Solver Program Interface.

Solver programs for CVRP are implemented as step-wise selection functions invoked within a fixed greedy route-construction procedure. At each decision step, given the current node $v \in \{0, 1, \dots, N\}$ (with $v = 0$ denoting the depot), the depot index $v_0 = 0$, the feasible unvisited customer index collection U (a subset of $\{1, \dots, N\}$), the remaining vehicle capacity q , the customer demand vector $\mathbf{d} \in \mathbb{R}^{N+1}$ (with $d_0 = 0$), and the distance matrix $D \in \mathbb{R}^{(N+1) \times (N+1)}$, the solver program selects the next node to visit:

$$i^* = \text{select}(v, v_0, U, q, \mathbf{d}, D).$$

The decoding procedure enforces feasibility by restricting U to customers whose demand does not exceed the remaining capacity q . If no feasible customer exists, the current route is terminated and the vehicle returns to the depot. Only the selection function varies across solver programs; all other components of the route-construction procedure are fixed.

G.3. Prompt Configuration

We specify task-level base prompts for the capacitated vehicle routing problem, as illustrated in Figure 10. These prompts define the CVRP task, including the optimization objective, required function interfaces, and task-specific constraints for solver and generator synthesis. Prompt construction for CVRP follows the same procedure as in online bin packing. Specifically, the CVRP base prompts are embedded into the unified prompt template structure shown in Figure 5, with task-specific content populated accordingly and all remaining components following the standard template of the search mechanism.

G.4. Evolution Outcomes

Figure 11 visualizes representative solver and generator programs discovered by ASRO on CVRP. On the solver side, the learned rules incorporate savings- and regret-shaped customer selection under explicit capacity pressure. On the generator side, instance constructions evolve from simple sampling to spatially clustered geometries with heterogeneous demand profiles that stress routing decisions.

G.5. Benchmarks (CVRPLIB).

We evaluate CVRP solvers on benchmark instances from *CVRPLIB* (Uchoa et al., 2017), a widely used public repository of capacitated vehicle routing problems. CVRPLIB contains instances with diverse spatial distributions, demand patterns, and capacity constraints, and is commonly adopted for evaluating both heuristic and learning-based routing algorithms.

We consider instance families A, B, E, F, M, P, and X. These families span a broad range of problem sizes and structural

Figure 10. Base prompt examples for solver and generator synthesis in the Capacitated Vehicle Routing Problem (CVRP).

Prompt for Generator

Task.

Write a novel Python function that produces realistic, well-structured CVRP instances designed to reduce the expected solution quality achieved by a mixed pool of heuristic solvers. The difficulty should arise from general structural properties of customer distributions, demand patterns, and capacity constraints, rather than solver-specific tricks.

The generation process may follow a structured protocol: (A) choose a sampling plan that induces hard solution landscapes (e.g., clustered customers, uneven demands, tight capacity); (B) sample realistic customer locations, avoiding extreme clustering or purely uniform layouts; (C) assign demands that create non-trivial capacity challenges via a mix of small and large demands; (D) perform lightweight self-checks (e.g., demand sum, capacity feasibility, spatial spread), and resample with bounded attempts if degenerate cases arise.

I/O.

Implement `generate_instances(seeds, num_customers, vehicle_capacity)` and return instances.

Input/Output Info.

`seeds`: iterable of integers, one per instance.
`num_customers`: number of customers per instance (int).
`vehicle_capacity`: vehicle capacity (int).
`instances`: Python list with the same length as `seeds`; each element is a dict with exactly the following keys: `depot`: list $[x, y]$ representing depot coordinates; `customers`: list of dicts, each with `'coords'` $[x, y]$ in $[0, 100] \times [0, 100]$ and `'demand'` (positive integer); `vehicle_capacity`: integer.

Other Info.

Avoid pathological or degenerate instances; prefer bounded resampling with quick checks over heavy computation. Output ONLY the Python function code.

Prompt for Solver

Task.

Write a step-by-step Python construction heuristic for the CVRP that selects the next node at each step to minimize total travel distance while respecting vehicle capacity constraints. The heuristic should aim to perform well across instances drawn from a mixture of generators, rather than overfitting to a single instance distribution.

I/O.

Implement `select(current_node, depot, unvisited_nodes, rest_capacity, demands, distance_matrix)` and return `next_node`.

Input/Output Info.

`current_node`: int, current location (0 denotes the depot; $1..n$ are customers).
`depot`: int, always 0.
`unvisited_nodes`: NumPy array of feasible customer IDs (subset of $\{1, \dots, n\}$).
`rest_capacity`: remaining vehicle capacity (float).
`demands`: NumPy array of length $(n + 1)$ with `demands[0]=0`.
`distance_matrix`: NumPy array of shape $(n + 1, n + 1)$ with symmetric distances.
`next_node`: int, either 0 (return to depot) or an element of `unvisited_nodes`.

Other Info.

Output ONLY the Python function code.

characteristics (Uchoa et al., 2017). In particular, A and B instances are relatively regular and homogeneous, while E, F, M, P, and X exhibit increasing heterogeneity in customer distributions, route lengths, and demand structures.

All CVRPLIB instances used for evaluation are distinct from the synthetic distributions employed during training. Performance is measured using normalized optimality gaps with respect to reference solutions, providing a standardized and widely accepted evaluation protocol.

H. Additional Ablation Results

This appendix reports detailed ablation results that support the analysis in Section 4.4. Due to space constraints, these results are omitted from the main text.

H.1. Data Augmentation and Self-play

TSP Data augmentation distribution. The data augmentation (DA) training distribution is constructed as a fixed mixture of five geometric families with weights: uniform square (30%), clustered Gaussian (25%), grid-jitter (15%), annulus/ring (15%), and two-scale mixture (15%). For each instance, a family is sampled according to these weights, followed by family-specific coordinate generation. Full specifications are given below.

(A) Uniform square (30%). Each city is sampled independently as $(x, y) \sim \text{Unif}([0, 1]^2)$.

(B) Clustered Gaussian (25%). The number of clusters is sampled as $k \sim \text{Unif}\{3, \dots, 8\}$, with cluster centers drawn uniformly in $[0, 1]^2$. Each city is assigned to a random cluster and sampled as $(x, y) = c_j + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ and $\sigma \sim \text{Unif}[0.02, 0.08]$. Coordinates are clipped to $[0, 1]^2$.

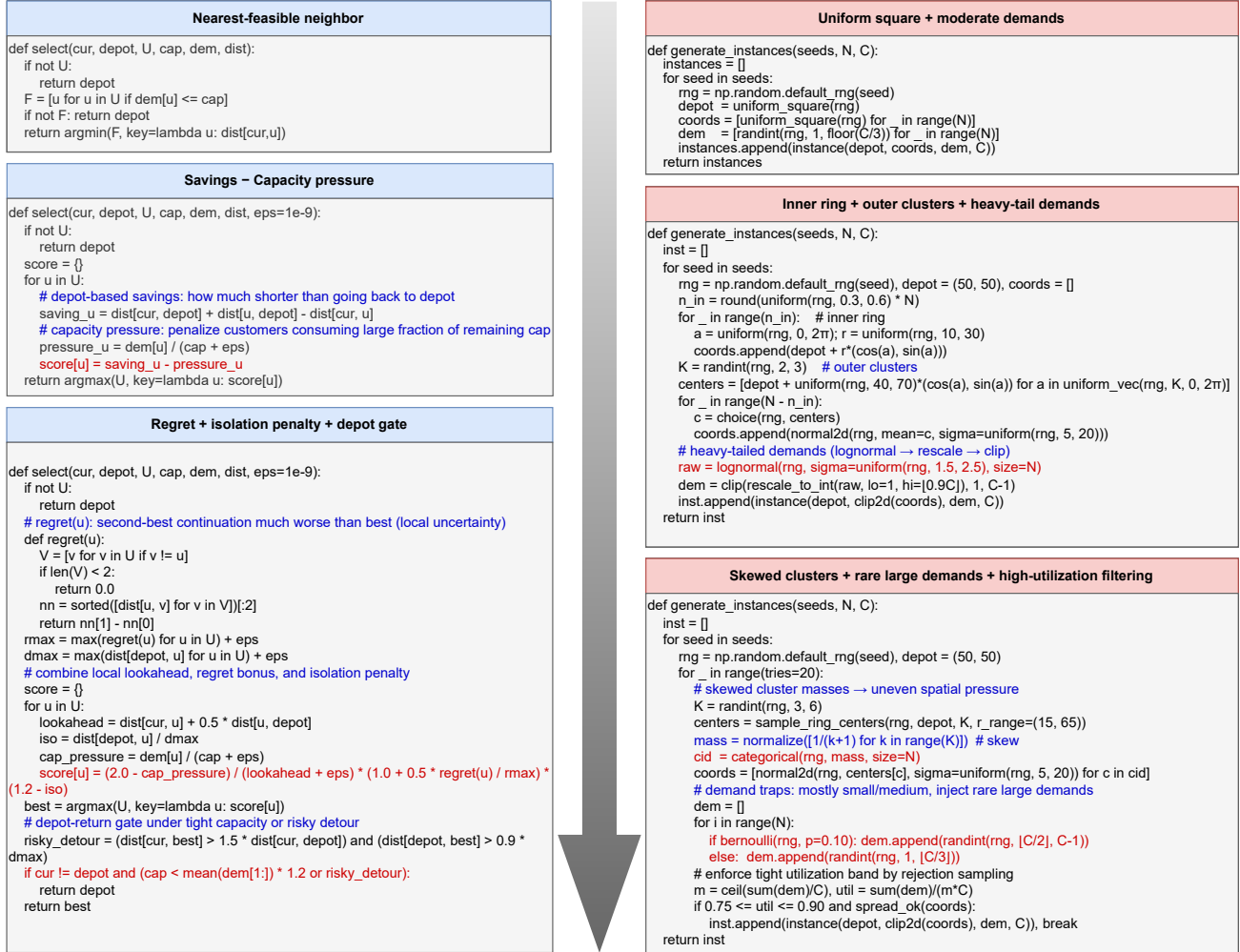


Figure 11. Representative solver and generator programs discovered by ASRO on the capacitated vehicle routing problem (CVRP). We show distilled solver customer-selection rules (left) and instance generators (right) drawn from the final solver and generator pools. Code is schematized to highlight the core algorithmic logic induced by adversarial pressure; auxiliary implementation details (e.g., batching, feasibility filtering, and acceptance criteria) are omitted for clarity.

(C) Grid-jitter (15%). An approximately $\lceil \sqrt{n} \rceil \times \lceil \sqrt{n} \rceil$ regular grid is constructed over $[0, 1]^2$, where n denotes the number of cities. Grid points are perturbed by i.i.d. noise $\delta_x, \delta_y \sim \text{Unif}[-0.05, 0.05]$, and n points are selected; if necessary, remaining points are sampled uniformly.

(D) Annulus / ring (15%). Angles are sampled uniformly in $[0, 2\pi]$ and radii are sampled uniformly from $[r_0, r_1]$, where $r_0 \sim \text{Unif}[0.2, 0.4]$ and $r_1 \sim \text{Unif}[0.5, 0.7]$. Polar coordinates are converted to Cartesian coordinates, translated to the unit-square center, and clipped to $[0, 1]^2$.

(E) Two-scale mixture (15%). Seventy percent of cities are sampled uniformly over $[0, 1]^2$, while the remaining 30% are sampled within a small square of side length sampled from $[0.05, 0.15]$ at a random corner. City order is randomly permuted after generation.

Self-play baseline. The self-play variant removes the meta-game structure of ASRO while retaining alternating solver-generator optimization. At each iteration, the solver and generator are updated only against their most recent counterparts, without maintaining a strategy pool or computing mixed meta-strategies. Concretely, self-play is implemented by restricting both the solver and generator strategy pools to size one, so that only the most recent program is retained on each side. This results in a greedy, memoryless adversarial loop rather than equilibrium-based co-evolution.

Table 10. **Equal-time comparison between EoH and ASRO-EoH on TSP.** Both methods are evaluated under the same wall-clock time budget (2200s).

Method	LIB-S	LIB-M	LIB-L	LIB-XL	Uniform
EoH	0.769	1.985	3.399	4.201	0.270
ASRO-EoH	0.319	0.936	2.417	3.993	0.117

Table 11. **Computational cost comparison between ASRO and EoH on TSP.** Both methods are evaluated under the same wall-clock time budget (12,000s). Here $T = 8$, $R = 2$, $K = 50$, $G = 50$, $n_{\text{inst}} = 3$, and $|\text{supp}(\sigma)| = 1.5$ denotes the average support size of the opponent meta-strategy.

Cost Metric	ASRO	EoH
LLM API Calls	$2(1 + RT)K = 1700$	$(1 + G)K = 2550$
Program Evaluations	$(2(1 + RT)K \text{supp}(\sigma) + (T + 1)^2)n_{\text{inst}} = 7893$	$(1 + G)Kn_{\text{inst}} = 7650$

H.2. Effect of LLM Backbones

Backbone specification. We evaluate ASRO with multiple LLM backbones for program-space search: DeepSeek-V3.2 (Liu et al., 2025a), Gemini-3-Flash-Preview, GPT-5-Nano, GPT-5-Mini, and LLaMA-3-8B-Instruct (Grattafiori et al., 2024). All models are used with identical prompts and decoding configurations.

H.3. Oracle-agnosticity

ReEvo (Ye et al., 2024) is an evolutionary program-synthesis framework that integrates LLMs into evolutionary search by using LLMs both to generate new heuristic candidates and to produce reflective feedback that guides evolution. Compared to the EoH-style program search mechanism, which relies on implicit feedback from heuristic evaluations, ReEvo explicitly incorporates reflective LLM feedback into the evolutionary loop via short- and long-term reflections that guide crossover and mutation. In this ablation, the EoH-style program search mechanism used in the main experiments is replaced with a ReEvo-style mechanism to instantiate ASRO’s program-space best-response oracle.

H.4. Computation Cost Details

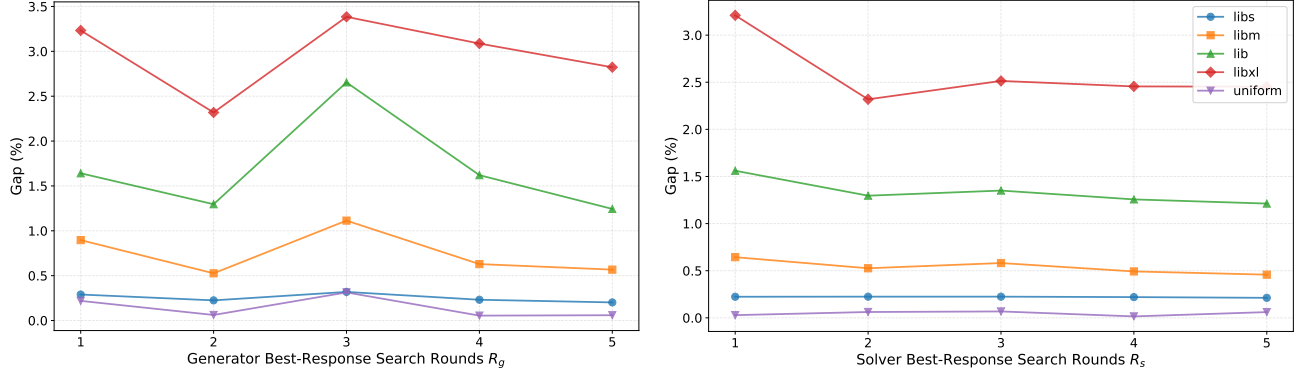
This section provides additional details for the computation cost analysis. Both ASRO and the EoH baseline are evaluated under matched wall-clock budgets using the same batch evaluation engine described in Appendix C. Table 10 reports the equal-budget comparison under a 2200-second budget, where ASRO consistently achieves lower optimality gaps across all benchmark regimes.

Table 11 further reports a detailed computational cost breakdown under a larger 12,000-second budget. The results show that, despite performing repeated solver–generator interaction evaluations and best-response searches on both sides of the game, ASRO uses fewer LLM API calls than EoH under the same wall-clock budget. The additional overhead of ASRO primarily arises from program evaluations, which are highly parallelizable in practice.

I. Additional Ablation Studies

Ablation on Best-Response Search Depth We observe heterogeneous sensitivity across benchmarks. Simpler instance distributions (e.g., uniform instances) are largely insensitive to best-response depth, while more structured benchmarks (e.g., TSPLIB-XL) exhibit clearer responses. Increasing solver- or generator-side rounds generally improves best-response accuracy, but with diminishing returns. On the solver side, performance largely saturates at $R_s = 2$. On the generator side, moderate depth is sufficient; isolated fluctuations may arise due to stochastic effects and conditioning on a fixed counterpart during ablation. Overall, we adopt $R_s = 2$, $R_g = 2$ as a balanced setting that achieves strong performance with substantially lower computational cost.

Ablation on Base-Generator Mixing Ratio. The minimum ratio specifies a lower bound on the probability mass assigned to the base generator, with the remaining mass allocated to the learned generator mixture during solver-side best-response search. Its effect varies with instance difficulty. For simpler distributions, performance is largely insensitive to this parameter, as the solver can reliably optimize against the base generator. In contrast, more structured benchmarks exhibit a clearer



(a) Varying generator best-response rounds R_g with solver rounds fixed ($R_s = 2$). (b) Varying solver best-response rounds R_s with generator rounds fixed ($R_g = 2$).

Figure 12. **Ablation of best-response search rounds in ASRO on Euclidean TSP.** Each subplot reports the optimality gap (%) across TSP benchmark instances under different configurations of solver-side and generator-side best-response rounds.

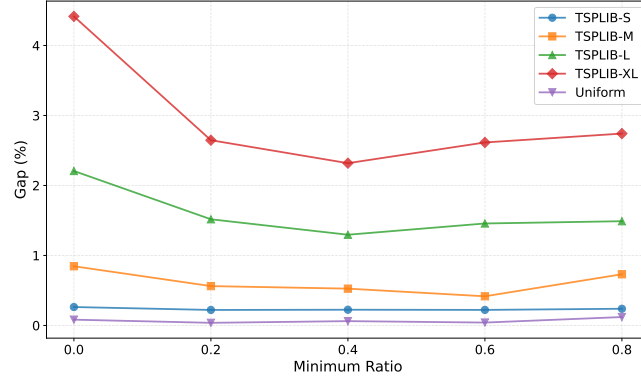


Figure 13. **Ablation of the minimum base-generator ratio in ASRO on Euclidean TSP.** Results report the average optimality gap (%) across TSP benchmarks under different minimum ratio settings.

dependence: very small ratios may lead to instability due to overfitting to the learned generator mixture, while excessively large ratios bias the opponent distribution toward the base generator and limit the effectiveness of co-evolution.